

Adaptive Self-Deceiving Honeypot

ABSTRACT

Static honeypots provide valuable visibility into adversarial behavior but remain inherently predictable, limiting their effectiveness against automated and behavior-aware attackers. Although recent research has explored adaptive deception using reinforcement learning and multi-layer architectures, limited work integrates reproducible supervised threat modeling with state-aware deception control under structured experimental validation. This paper presents an Adaptive Self-Deceiving SSH Honeypot that couple session-level behavioral classification with a deterministic yet dynamically evolving deception engine. The system is implemented using a Cowrie-based SSH honeypot deployed in an isolated environment. Attacker interactions are transformed into an 18-dimensional session-level feature space capturing command frequency, reconnaissance activity, privilege escalation attempts, sensitive access patterns, download behavior, and temporal characteristics. A four-class supervised classifier (LOW, MEDIUM, HIGH, CRITICAL) is trained on 3,189 real labeled sessions using a stratified protocol. The best-performing XGBoost model achieves 90.18% accuracy and a macro F1-score of 0.8818 on a held-out test set. At runtime, classification outputs drive a DECEIVE engine that enforces progressive defensive states (Normal, Alert, Hardened, Containment) using hysteresis-controlled transitions and deterministic hard-trigger escalation for destructive commands. In a structured 540-session comprehensive evaluation, the system achieves 71.30% overall accuracy and 0.7351 macro F1-score while maintaining 100% hard-trigger detection and 100% correct state-transition enforcement, yielding an operational success rate of 88.5%. The results demonstrate that combining supervised behavioral modeling with state-aware deception provides a reproducible and operationally safe adaptive honeypot framework. While boundary ambiguity between escalation levels and early-stage window classification remains open challenges, the proposed architecture establishes a measurable foundation for sequence-aware and semantically enriched cyber deception systems.

KEYWORDS

Adaptive Honeypot, Cyber Deception, Behavioral Threat Classification, State-Aware Defense, XGBoost, Attacker Behavior Modeling, Intrusion Intelligence.

1. Introduction

The increasing automation and behavioral sophistication of cyberattacks continue to challenge conventional defensive mechanisms. Modern systems are persistently exposed to automated scanning, credential brute-forcing, opportunistic exploitation, and post-exploitation reconnaissance. While firewalls and intrusion detection systems (IDS) remain essential components of defensive architectures, they primarily focus on detection and blocking of known signatures or anomalous patterns. Consequently, they offer limited insight into attacker intent, strategy evolution, and multi-stage behavioral progression. This lack of contextual intelligence restricts defenders' ability to understand how adversaries explore systems, escalate privileges, and adapt tactics over time.

Honeypots provide a complementary approach by intentionally exposing decoy systems to adversaries in controlled environments, thereby enabling detailed observation of malicious activity without risking production assets. Unlike preventive controls, honeypots emphasize behavioral monitoring and intelligence collection. However, most traditional honeypot deployments remain static in design, responding deterministically to attacker inputs. Such predictability makes them susceptible to fingerprinting through response timing analysis, environmental inconsistencies, or behavioral anomalies. As a result, skilled attackers may disengage prematurely, reducing both engagement duration and intelligence yield.

To address these limitations, this work proposes an Adaptive Self-Deceiving SSH Honeypot that integrates supervised behavioral classification with a state-aware deception engine. The system is implemented using a Cowrie-based SSH honeypot deployed within an isolated environment to ensure containment and operational safety. Attacker interactions including authentication attempts, executed commands, timing characteristics, and file-related actions are recorded in structured logs and transformed into session-level behavioral features.

A four-class supervised threat classifier (LOW, MEDIUM, HIGH, CRITICAL) is trained using an 18-dimensional feature space capturing command intensity, reconnaissance activity, privilege escalation attempts, sensitive access patterns, download behavior, and temporal statistics. Rather than using classification outputs for blocking or mitigation, the proposed system leverages them to dynamically control deception policies. A dedicated DECEIVE engine operates as a state-aware controller that modifies command outputs, injects controlled delays, simulates defensive failures, and applies deterministic escalation policies across progressive defensive states (Normal, Alert, Hardened, Containment). Hysteresis-controlled transitions and hard-trigger rules ensure both adaptive realism and safety enforcement.

The objective of this adaptive design is not immediate attack prevention but sustained adversarial engagement and structured intelligence collection. By maintaining believable system behavior while progressively adapting to attacker activity, the honeypot encourages deeper behavioral exposure without compromising containment. This architecture establishes a reproducible framework for combining behavioral modeling with operational cyber deception, advancing beyond static honeypot deployments toward measurable and state-aware adaptive interaction.

2. Research Objectives

The primary objective of this research is to design, implement, and evaluate a reproducible adaptive honeypot framework that integrates supervised behavioral modeling with state-aware deception control. Specifically, the study aims to:

1. Design and deploy an isolated SSH-based honeypot environment using Cowrie to safely capture real attacker interactions under controlled experimental conditions.
2. Construct a structured behavioral dataset by collecting session-level logs including authentication attempts, executed commands, temporal patterns, and file access activity, ensuring reproducibility and traceability.
3. Engineer a session-level feature representation that captures behavioral intensity, reconnaissance activity, privilege escalation attempts, sensitive access patterns, download/exfiltration indicators, and timing characteristics within an 18-dimensional feature space.
4. Develop and evaluate supervised threat classification models capable of categorizing attacker sessions into four threat levels (LOW, MEDIUM, HIGH, CRITICAL) using stratified training and rigorous performance validation.
5. Implement a state-aware DECEIVE engine that dynamically adapts honeypot responses across progressive defensive states (Normal, Alert, Hardened, Containment) using classification outputs, hysteresis-based transitions, and deterministic hard-trigger escalation.
6. Quantitatively evaluate system performance in terms of classification reliability, state-transition correctness, hard-trigger enforcement, and operational success under structured comprehensive testing scenarios.

Through these objectives, the research seeks to establish a practical and experimentally validated foundation for adaptive cyber deception systems that balance engagement realism, behavioral intelligence extraction, and operational safety.

3. Literature Review

Honeypots have historically served as a practical deception primitive for observing adversarial behavior, collecting malware artifacts, and understanding intrusion workflows in controlled environments. Spitzner's foundational text formalized honeypots as high-value observational assets and positioned them as intelligence-generating systems rather than purely preventive controls, establishing the conceptual basis for modern deception research [1]. However, the increasing sophistication of adversaries and the prevalence of automated reconnaissance have reduced the effectiveness of static honeypots, motivating a shift toward adaptive deception systems that can dynamically adjust services, responses, and engagement strategies over time.

Early work on adaptive deception emphasizes the need for realism and variability to counter honeypot fingerprinting. Iyer frames adaptive honeypots as dynamic deception mechanisms

capable of modifying configuration and interaction behavior to maintain authenticity and improve threat intelligence capture in modern cyber defense settings [2]. While the paper offers a clear conceptual argument and outlines challenges and opportunities, a key limitation is its lack of empirical evaluation: it does not provide a benchmark dataset, testbed validation, or quantified improvement measures over conventional honeypots [2]. As a result, the work is primarily useful for defining motivation and high-level design requirements rather than serving as evidence of measurable performance gains.

To operationalize adaptation, reinforcement learning (RL) has become a recurring technique, particularly for controlling how a honeypot responds to attacker actions and deciding when to reveal additional system functionality. Dowling *et al.* propose an intelligent honeypot that uses RL to interact with automated malware, selecting actions under a defined state-action formalism to optimize information collection [3]. Their contribution is valuable because it reframes honeypot interaction as a sequential decision problem, aligning deception control with learning-driven optimization rather than fixed scripting. The main loophole in this direction is that RL behavior is only as strong as its reward design and state representation, over-simplified state spaces risk producing policies that perform well in constrained environments but fail under diverse attacker strategies, long-horizon campaigns, or adversaries that intentionally probe for automation patterns [3]. Additionally, studies of malware-focused engagement may not transfer directly to human operators conducting interactive post-exploitation and lateral movement, which is a key requirement for realistic attacker behavior modeling.

In a similar RL-driven adaptation category, Pauna *et al.* introduce QRASSH, a self-adaptive SSH honeypot using Q-learning to choose how it should interact with external attackers [4]. The relevance of QRASSH lies in demonstrating a concrete adaptive mechanism for one of the most heavily targeted services (SSH), where attacker interactions are naturally sequential and measurable. Nonetheless, its scope is service-specific, which can be a limitation for broader deception frameworks that aim to simulate full enterprise environments. In practice, realistic attacker behavior frequently involves credential access, privilege escalation, internal discovery, and pivoting across multiple hosts and services, a single-service honeypot may collect partial behavior but not capture complete attack chains.

A more defensive-robustness perspective is introduced by Touch and Colin through Asguard, which explicitly combines engagement and self-protection. The core idea is to learn attacker interaction policies while guarding against deep compromise by integrating state environment and action into a reward design that balances intelligence gain and survivability [5], [6]. This is a meaningful advancement because traditional high-interaction honeypots face a persistent dilemma: the more realistic they become, the higher the risk that adversaries use them as stepping stones. The loophole is that protection constraints may reduce realism if the system becomes overly cautious, potentially revealing defensive intent. Moreover, evaluation realism can be difficult: demonstrating “self-guarding” effectiveness often requires careful containment assumptions and clear adversary capability definitions, which can vary widely across deployments.

Beyond engagement control, adaptive honeypots have also been explored for resilience against disruptive behaviors such as denial-of-service. Veluchamy and Kathavarayan present a deep reinforcement learning approach for building honeypots against runtime DoS attacks [7]. This line of work contributes toward maintaining availability and stability of deception systems under high-volume adversarial pressure. However, a consistent limitation in DoS-centric honeypot studies is that they often emphasize attack detection/classification performance more than deception believability. In attacker-behavior research, sustaining realism typically matters as much as resisting disruption, therefore, any model focusing only on classification gain without measuring adversary dwell time, interaction depth, and multi-stage progression may under-represent the full deception objective.

In constrained or domain-specialized networks, such as Internet of Battlefield Things (IoBT), Limouchi and Mahgoub investigate RL-assisted threshold optimization for dynamic honeypot adaptation, enabling dynamic switching decisions driven by learned thresholds [8]. The major contribution is showing that adaptive control mechanisms can be lightweight enough to operate in environments with limited resources and high mobility constraints. The loophole is the dependency on threshold-based decision abstractions: these decisions can be sensitive to environment-specific tuning and may oversimplify attacker behavior into coarse engagement triggers, limiting the fidelity of attacker modeling. For projects targeting realistic attacker behavior simulation, threshold adaptation should ideally be combined with richer behavioral features and multi-stage reasoning.

A parallel supporting direction involves improving detection and traffic classification pipelines so that deception resources can prioritize high-value interactions. Jadav *et al.* explore machine learning approaches to classify network traffic, supporting the broader idea that intelligent preprocessing can help deception systems decide when to escalate engagement [9]. However, traffic classification alone does not inherently model attacker intent or campaign structure, it primarily acts as a filter. Without combining classification outputs with behavior sequencing or attack-stage inference, the system may still struggle to approximate realistic attacker progressions and decision-making.

Recent work increasingly promotes multi-layer deception and selective escalation, where low-interaction sensors filter broad background noise and only valuable sessions are escalated into high-interaction environments. Möller proposes ADLAH, an adaptive multi-layer honeynet architecture for threat behavior analysis using deep learning and a reinforcement learning-based decision mechanism that escalates sessions from low-interaction nodes to dynamically provisioned high-interaction honeypots [10]. A strength of this approach is its pragmatic response to real-world conditions: exposed services often face heavy automated traffic, and high-interaction environments are costly and risky to run at scale. The important limitation is explicitly stated by the author: sufficient live data were unavailable, and field-scale validation is not claimed, thus, the major output is an architectural blueprint supported by a prototype rather than a fully benchmarked system [10]. For an academic project, this becomes a research opportunity: the architecture can be used, but careful evaluation design is still required to demonstrate measurable improvements in realism and intelligence yield.

At a higher abstraction level, cyber deception has also been studied through simulation and attack-graph reasoning, enabling defenders to formally evaluate effectiveness and optimize placement. Reti *et al.* propose a methodology to quantitatively evaluate deception (honeypots) and moving target defense (MTD) using network attack simulation, producing guidance on optimal deployment quantity and configuration intervals under varying network and attacker configurations [11]. This is a key contribution for projects that need defensible experimental methodology without relying entirely on risky live deployment. Its loophole is that simulation validity depends on how accurately attacker models, network constraints, and detection assumptions reflect reality, overly simplified attacker agents can produce misleading optima.

Complementing this, Sayed *et al.* study honeypot allocation for cyber deception in dynamic tactical networks using a game-theoretic model incorporating network mobility and a limited deception budget, with equilibrium-driven strategies validated through numerical simulations [12]. This work is directly relevant to the “where and when to deploy deception” question and aligns well with adversary path manipulation. The limitation, again, is the abstraction gap: while attack graphs and game models are powerful for decision-making, they often require assumptions about attacker goals and rationality. Real attackers do not always behave optimally, and automation/bot behavior may deviate significantly from the rational attacker models common in security games.

A broad survey perspective is provided by Javadpour *et al.*, who categorize cyber deception techniques aimed at improving honeypot performance and reducing detectability, with comparative discussion and modeling approaches [13]. This survey is useful for positioning an academic project in the research landscape and for identifying deception “failure modes” such as fingerprinting and attacker adaptation. The loophole with survey-heavy sources is that they synthesize rather than prove, the actionable value comes from translating surveyed techniques into measurable design requirements and evaluation metrics in an implemented system.

Finally, work from RL outside classical honeypot systems provides relevant methodology for attacker path shaping, which is central to realistic behavior simulation. Fatemi *et al.* study deceptive path planning using reinforcement learning with graph neural networks, addressing limitations of classical deceptive path planning methods and emphasizing generalization, scaling, tunable deception levels, and adaptivity to environmental changes [14]. While the domain is not strictly honeypots, it contributes a transferable framework for reasoning about deception objectives on graphs. The limitation for direct application is that mapping real enterprise networks and adversary behaviors into weighted graphs and defining deception rewards remains non-trivial, however, the paper is a strong conceptual bridge for systems that aim to guide attacker trajectories through deception-layer design.

Chelliah and Swamy examine deception from a strategic and decision-making perspective, arguing that deception is most effective when it is adaptive, goal-oriented, and responsive to an opponent’s behavior rather than static or rule-based. Although their work is grounded in business strategy rather than cybersecurity, the core insight is applicable to cyber deception systems, where defenders must dynamically influence attacker perception and actions [15]. The

limitation of this work lies in its conceptual nature, as it does not provide algorithmic models or experimental validation suitable for direct technical implementation.

Masters and Sardina introduce the concept of deceptive path planning, where an agent deliberately selects misleading actions to influence an observer's belief about the agent's true goal. This work formalizes deception as a planning problem and demonstrates how deceptive behavior can be systematically generated rather than scripted. While originally applied to planning domains, the concept directly maps to cyber deception, where attackers can be guided along false interaction paths [16]. However, the study assumes well-defined observer models, which may not fully capture the unpredictability of real-world attackers.

Price et al. extend deceptive planning through a domain-independent framework, enabling deception strategies to be applied without reliance on domain-specific heuristics. This generalization is valuable for adaptive cyber deception systems, as it supports reusable deception logic across different environments and attack scenarios [17]. A key limitation is that evaluation is primarily simulation-based, and translating abstract planning models into operational cyber systems requires additional assumptions about attacker behavior and system observability.

Huang and Zhu investigate deceptive reinforcement learning under adversarial manipulation of cost signals, demonstrating that learning agents can maintain deceptive policies even when feedback is noisy or adversarially influenced. This is particularly relevant to cyber environments, where attackers may intentionally probe system responses to infer defensive mechanisms. The main drawback of this approach is the reliance on controlled experimental settings, leaving questions about scalability and robustness in live cyber deployments [18].

Lewis and Miller explore deceptive reinforcement learning in model-free domains, showing that deceptive behavior can emerge without an explicit model of the adversary. This finding supports the feasibility of adaptive deception engines that respond dynamically to attacker actions based solely on interaction history. However, the absence of explicit adversary modeling may reduce interpretability and make it difficult to reason about why certain deceptive behaviors emerge [19].

Liu et al. propose deceptive reinforcement learning for privacy-preserving planning, where agents intentionally mislead observers to conceal sensitive objectives while still achieving task goals. This aligns closely with cyber deception objectives, where real assets and defensive capabilities must remain hidden behind decoy behaviors. The limitation of this work is that it focuses primarily on privacy preservation rather than long-term adversarial engagement or intelligence extraction [20].

Savas et al. study deceptive planning for resource allocation, addressing how deception strategies can be optimized under limited resources and operational constraints. This is particularly relevant for cloud-based honeypot deployments, where high-interaction environments are costly and must be allocated selectively. The work, however, assumes rational adversaries and structured environments, which may not fully represent opportunistic or automated attackers commonly observed in real networks [21].

Savas et al. further extend this line of research by examining deceptive decision-making under uncertainty, proposing methods that maintain deceptive effectiveness when system states and adversary beliefs are partially observable. This is highly applicable to real-world cyber deception, where defender knowledge of attacker intent is inherently incomplete. A key limitation is that the evaluation remains largely theoretical and simulation-driven, highlighting the need for empirical validation in live or semi-live environments [22].

Recent advances in cyber deception have progressively shifted from static honeypot deployments toward hybridized, adaptive, and machine learning–driven architectures. Qin *et al.* [23] provide a comprehensive taxonomy of Honey-X–based hybrid cyber defense strategies, systematically categorizing the integration of honeypots, honeynets, and honeytokens with Defensive Cyber Deception (DCD), Moving Target Defense (MTD), Machine Learning (ML), and Game Theory (GT) across IoT, CPS, and industrial domains. While the survey establishes structural patterns and evaluation metrics such as survival rate, temporal effectiveness, and probabilistic cost models, it remains descriptive without empirical validation. Extending hybrid deception into operational industrial settings, Qin *et al.* [24] introduce a CGAN-based framework that dynamically reshapes ICS network topologies using Software-Defined Networking (SDN), demonstrating through simulation that generative adversarial modeling can reduce reconnaissance success while preserving system availability; however, the absence of real-world deployment and scalability validation highlights practical limitations. The emergence of LLM-driven honeypots marks a further evolution toward high-interaction deception systems. Malhotra [25] proposes LLMHoney, a hybrid SSH honeypot combining dictionary-based low-latency responses with large language model generation for novel commands, evaluated using semantic similarity metrics and latency analysis, though challenges such as hallucinated outputs and computational overhead remain. Similarly, Otal and Canbaz [26] demonstrate supervised fine-tuning of open-source LLMs to mimic realistic shell responses, improving engagement fidelity while remaining constrained by dataset dependency and resource cost. Comparative analysis of retrieval-augmented generation (RAG) versus prompt-tuned models in SBASH [27] further reveals a realism-latency trade-off, emphasizing the need for efficiency-aware generative deception. Providing structured synthesis, Bridges *et al.* [28] systematize LLM-honeypot integrations and identify gaps in scalability, evaluation consistency, and operational robustness. Beyond interaction realism, honeypot-driven anomaly detection approaches such as the Isolation Forest-based framework evaluated on the BETH dataset in *Symmetry* [29] demonstrate improved AUROC relative to baseline models, yet remain dataset-specific and detached from adaptive deception logic. Adaptive IoT-focused deception frameworks [30] propose dynamic behavioral adjustment based on observed attack patterns but lack large-scale empirical validation. At a strategic level, Park and Dagher [31] employ Bayesian Stackelberg games for honeypot allocation in multi-attacker networks, enabling scalable placement optimization though without integrating behavioral adaptation mechanisms. Wu *et al.* [32] introduce an active honeypot guardrail concept for detecting multi-turn LLM jailbreaks using bait-based probing and utility scoring, yet the approach is evaluated in controlled experimental settings rather than live network environments.

Singh [33] investigates SSH attacker behaviors by deploying a Cowrie honeypot and applying K-Means clustering to the captured interaction logs. Following feature extraction and normalization, the study groups similar sessions to uncover recurring attack patterns. This clustering approach transforms raw honeypot data into structured insights about attacker tactics, offering a way to segment malicious activity without labeled datasets. While effective in revealing behavioral clusters, the work is limited by its reliance on a fixed number of clusters and metric-based segmentation, which may not capture semantic nuances in command sequences or generalize across diverse threat environments. Sanjana [34] outlines the design of autonomous cyber defense systems that combine deception techniques with AI/ML for real-time detection and response. This architecture emphasizes adaptive decoy environments that evolve based on attacker interactions, enabling autonomous threat identification and dynamic defensive adjustments without human intervention. The approach highlights the potential of integrating learning-based models with deception logic to bridge detection and response. However, the chapter is largely conceptual, offering limited empirical evaluation and leaving implementation complexity, data dependency, and performance validation as open areas for further research. Recent research has increasingly framed honeypot deployment and detection as adaptive and learning-driven problems rather than static configuration tasks. Ouwehand [35] formulates honeypot detection and deployment as an adversarial reinforcement learning (RL) problem in a graph-structured network environment. The framework models both attacker and defender agents using Proximal Policy Optimization (PPO) with a shared Graph Neural Network (GNN) architecture to capture topology-aware host interactions. The attacker performs budget-constrained probing and classification actions, evaluated using confusion-matrix-based metrics including accuracy, true positives, false positives, and classification coverage. Experimental results demonstrate that when facing an adaptive defender, attacker classification accuracy drops to approximately 75%, while the Honeypot Detection Rate (HDR) stabilizes below 0.2, indicating reduced honeypot detectability under learned defensive strategies [35]. Although methodologically rigorous, the study is limited to a simulated environment and does not validate performance under real-world traffic conditions.

In contrast, Muntaha *et al.* [36] present an AI-powered cybersecurity framework integrating machine learning detection, adaptive response, and deception mechanisms within a simulation-based evaluation environment. Reported results indicate 97% machine learning accuracy, 1.8-second adaptive response time, 90% successful deception deployment, 93% real-time adaptation capability, 93% threat visibility, and 85% prevention success rate. These findings suggest strong simulated performance across multiple resilience metrics. However, the evaluation remains entirely simulation-driven and lacks adversarial stress testing against adaptive attackers, limiting conclusions regarding real-world robustness. The SES journal contribution [36] synthesizes adaptive honeypot architectures and AI-enhanced deception strategies, emphasizing hybrid deployment models and reinforcement learning integration. While the work provides a broad literature integration and conceptual framework for intelligent deception, it does not present detailed experimental validation or original empirical performance metrics, functioning primarily as a survey-oriented synthesis.

Recent work has advanced adaptive cyber deception through intelligent honeypot allocation and game-theoretic optimization. Hoang et al. [37] propose *Hawkeyes*, a hierarchical reinforcement learning framework that integrates stage-aware high-level defense strategies with fine-grained honeypot placement decisions, demonstrating improved trapping efficiency and adaptive responsiveness under simulated enterprise intrusion scenarios. Extending allocation modeling, Dumisani [38] formulates honeypot diversification as a game-theoretic redundancy optimization problem, ensuring system availability while increasing adversarial uncertainty through diversified deployment strategies. Similarly, Zambianco et al. [39] present a proactive decoy selection scheme that optimizes deception resource placement by anticipating adversarial tactics, enhancing resilience against multi-stage attack campaigns. Collectively, these studies emphasize intelligent honeypot allocation and strategic deception planning, yet primarily rely on simulation-based validation and focus on placement optimization rather than runtime behavioral manipulation or session-level adaptive response mechanisms.

Wang et al. [40] present a proactive cyber deception framework that combines Bayesian attack graphs with Stackelberg game theory to optimize honeypot allocation under uncertainty. Their model captures probabilistic attacker progression and formulates defense strategy selection as a leader-follower game, enabling risk-aware and stage-sensitive deception planning. While the approach improves strategic defense effectiveness compared to baseline allocation methods, it focuses on high-level planning and does not address session-level behavioral adaptation or real-time deception dynamics. Recent advances in cyber deception have increasingly addressed the challenge of defending against AI-enabled and autonomous adversaries. Kouremetis et al. [41] introduce *Mirage*, a deception framework designed to counter autonomous cyberattacks by dynamically adapting decoy behavior to AI-driven reconnaissance strategies, demonstrating that deception must evolve to match the decision-making capabilities of machine-driven attackers. Expanding foundational understanding, Rogers [42] presents the *Tularosa Experiment*, a structured empirical study examining deception efficacy across controlled adversarial engagements, establishing baseline methodologies for evaluating deception performance beyond traditional honeypot metrics. Letychevskyi [43] proposes personalized adaptive signaling strategies grounded in cognitive modeling, arguing that deception effectiveness depends not only on technical artifacts but also on attacker perception dynamics. However, most studies emphasize strategic allocation or theoretical modeling and remain limited in session-level behavioral adaptation or real-world validation against sustained interactive attack traffic.

Overall, the literature indicates a clear evolution from static honeypots toward learning-driven adaptive deception ecosystems. Despite promising architectural models and algorithmic control strategies, common unresolved gaps persist across studies: limited field-scale validation, sensitivity to reward and state design in RL-based control, narrow service scope in many implementations, and incomplete measurement of realism beyond detection or classification accuracy. These gaps define a strong research space for student projects that combine multi-layer honeynets, adaptive engagement, attacker-stage modeling, and realistic

activity generation while grounding evaluation in either safely contained testbeds or defensible simulation-based methodologies.

4. System Architecture

The proposed Adaptive Self-Deceiving Honeypot is designed as a modular, end-to-end pipeline that integrates behavioral data acquisition, supervised threat classification, and state-aware deception control within a controlled and isolated execution environment. The architecture is structured to ensure reproducibility, operational containment, and clear separation between offline model training and runtime deception orchestration.

4.1 Architectural Overview

The architecture consists of two primary layers:

1. Offline Data and Model Layer, responsible for dataset construction, feature engineering, and classifier training.
2. Runtime Deception Layer, responsible for real-time session monitoring, threat classification, state transitions, and adaptive response generation.

This separation ensures that model development and evaluation are performed independently from live deception execution, thereby reducing runtime complexity and improving experimental rigor.

4.2 Data Acquisition and Logging

At the interaction layer, the system employs a Cowrie-based SSH honeypot deployed within an isolated environment to ensure strict containment. All attacker interactions including authentication attempts, executed commands, timestamps, failed commands, and session termination events are recorded in structured JSON logs.

These logs form the primary data source for both offline dataset construction and runtime monitoring. Structured logging ensures consistency between training and deployment pipelines while maintaining traceability of session-level behavior

4.3 Offline Behavioral Modeling Layer

The offline layer transforms raw interaction logs into session-level behavioral representations. Events are grouped by session identifier and converted into an 18-dimensional feature vector capturing command intensity, reconnaissance activity, privilege escalation attempts, sensitive access patterns, download behavior, and temporal characteristics.

The processed dataset is used to train a four-class supervised threat classifier (LOW, MEDIUM, HIGH, CRITICAL). Model training is performed using stratified session-level splits to avoid data leakage. The resulting trained model artifact includes the classifier, feature list, and optional scaling components, ensuring consistency during runtime inference.

This layer produces a serialized model artifact that is loaded by the runtime engine without requiring retraining or recalibration. Offline behavioral modeling layer of the Adaptive Self-

Deceiving SSH Honeypot. Raw interaction logs generated by the Cowrie SSH honeypot are aggregated at the session level and transformed into an 18-dimensional behavioral feature representation through structured extraction and preprocessing. These features are used to train a supervised four-class threat classifier (LOW, MEDIUM, HIGH, CRITICAL), producing a serialized model artifact for deployment. This layer operates independently of runtime execution to ensure reproducible dataset construction, stratified evaluation, and controlled model development.

4.4 Runtime Monitoring and Session Management

During deployment, a log-monitoring component continuously tails the honeypot's structured output and parses relevant events. A session manager maintains per-session state, including command history, timestamps, behavioral counts, and prior threat classifications.

Feature extraction is triggered when the session command buffer reaches a predefined window size (default: five commands). This window-based approach enables early-stage threat assessment while preserving computational efficiency.

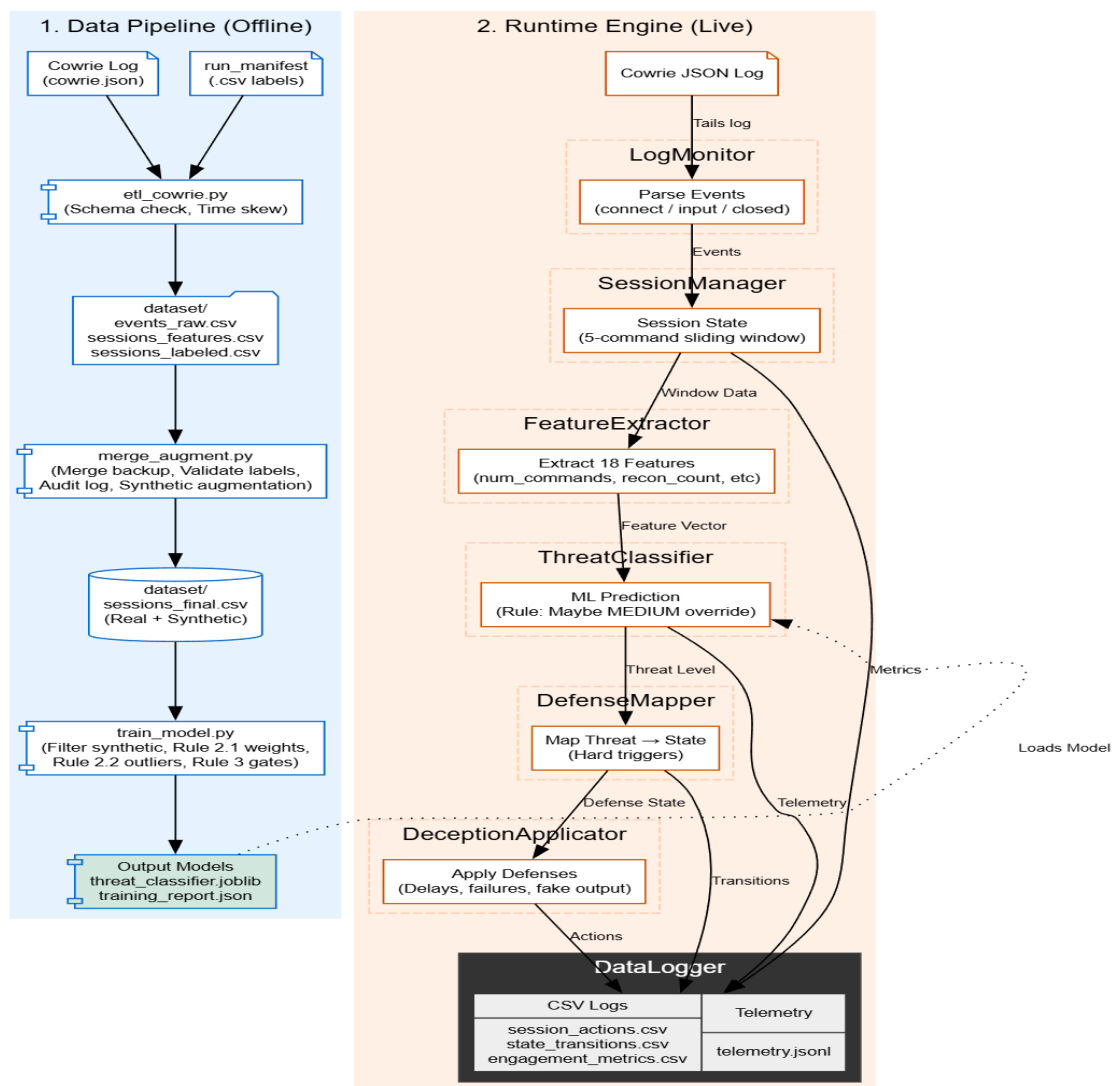


Fig. 1: System Architectural flow of offline pipe line and Runtime ETL.

Fig. 1. Architectural schematic of the DECEIVE framework, delineating the Offline Data Pipeline and the Live Runtime Engine. The offline phase (left) manages the extraction, transformation, and loading (ETL) of raw Cowrie logs, followed by synthetic data augmentation and model training to generate the threat_classifier. The runtime phase (right) orchestrates real-time defense: the SessionManager aggregates events into a sliding window for the FeatureExtractor, enabling the ThreatClassifier to predict attacker intent. These predictions drive the DefenseMapper to dynamically modulate the system state and trigger adaptive countermeasures via the DeceptionApplicator.

4.5 Threat Classification and State Mapping

Extracted feature vectors are passed to the trained classifier to obtain a threat category and associated probability distribution. Classification outputs are then mapped to progressive defensive states:

- LOW → Normal
- MEDIUM → Alert
- HIGH → Hardened
- CRITICAL → Containment

State transitions are governed by hysteresis thresholds to prevent oscillation and ensure behavioral stability. In addition, deterministic hard-trigger rules immediately escalate sessions containing destructive command patterns (e.g., reverse shells or file deletion attempts) to Containment, overriding probabilistic classification when necessary.

4.6 Deception Engine and Response Adaptation

The DECEIVE engine operates as a state-aware response controller that dynamically modifies honeypot behavior according to the current defensive state. Adaptation mechanisms include:

- Controlled response delays
- Simulated permission failures
- Deceptive command outputs
- Contextual security-theater messaging

These mechanisms are designed to increase attacker engagement duration while preserving operational safety and preventing real system compromise. Deception policies are applied at the command level, enabling fine-grained interaction control rather than coarse session blocking.

4.7 Operational Safety and Isolation

The entire architecture is deployed within a contained environment to prevent lateral movement or external misuse. The honeypot does not execute real attacker payloads, and no production

data is exposed. Hard-trigger escalation and containment policies ensure that high-risk behaviors remain strictly confined.

Runtime deception layer of the Adaptive Self-Deceiving SSH Honeypot. Live interaction logs are monitored and grouped by session, where window-based feature extraction (window size = 5) enables progressive threat inference. Classification outputs are mapped to defensive states (Normal, Alert, Hardened, Containment) using hysteresis-controlled transitions and deterministic hard-trigger escalation for high-risk commands. The DECEIVE engine then applies command-level adaptive responses, including controlled delays, simulated failures, and deceptive outputs, enabling real-time behavioral adaptation while preserving operational containment.

4.8 Architectural Contributions

The proposed architecture contributes a hybrid framework that combines:

- Supervised behavioral threat modeling,
- Deterministic safety enforcement through hard triggers, and
- State-aware adaptive deception at the interaction level.

Unlike purely reinforcement-learning-driven approaches, this design emphasizes reproducibility, interpretability, and measurable evaluation under controlled conditions. By clearly separating offline modeling from runtime deception orchestration, the architecture enables structured experimentation while maintaining operational realism.

5. Methodology

The proposed Adaptive Self-Deceiving Honeypot integrates supervised session-level threat modeling with state-aware deception control. The methodology consists of four principal stages: (i) session construction and feature extraction, (ii) supervised threat classification, (iii) runtime window-based inference, and (iv) state-aware deception mapping.

5.1 Problem Formulation

An attacker interaction session is represented as an ordered sequence of command–timestamp pairs:

$$S = \{(c_1, t_1), (c_2, t_2), \dots, (c_n, t_n)\} \quad (1)$$

where c_i denotes the i -th command issued by the adversary and t_i its corresponding timestamp, satisfying $t_1 < t_2 < \dots < t_n$.

The objective of the system is to infer a session-level threat label

$$y \in \{L, M, H, C\},$$

corresponding to LOW, MEDIUM, HIGH, and CRITICAL behavioral categories, and to map this label to an adaptive defensive state governing deception behavior.

Threat estimation is formulated as a supervised multi-class classification problem:

$$f_{\theta}: \mathbb{R}^{18} \rightarrow \{L, M, H, C\}, \quad (2)$$

where θ denotes the learned model parameters.

5.2 Threat Model and Assumptions

The adversary interacts with the honeypot through SSH and may be either automated or human-operated. The defender observes all issued commands and timestamps but does not directly observe attacker intent. The environment is fully sandboxed; no real payload execution or lateral movement is permitted. All responses are emulated.

The system therefore operates under partial observability of adversarial objectives but complete visibility of interaction traces within the controlled domain.

5.3 Session-Level Feature Representation

Each session is transformed into an 18-dimensional feature vector capturing temporal intensity, semantic command categories, and interaction ratios.

5.3.1 Temporal Features

Session duration:

$$D = \max(t_n - t_1, 1) \quad (3)$$

Command rate (commands per minute):

$$\text{CPM} = \frac{n}{D} \times 60 \quad (4)$$

Inter-command delays:

$$\Delta_i = t_{i+1} - t_i, i = 1, \dots, n - 1 \quad (5)$$

Mean delay:

$$\bar{\Delta} = \frac{1}{n - 1} \sum_{i=1}^{n-1} \Delta_i \quad (6)$$

Variance:

$$\sigma_{\bar{\Delta}}^2 = \frac{1}{n-1} \sum_{i=1}^{n-1} (\Delta_i - \bar{\Delta})^2 \quad (7)$$

5.3.2 Behavioral Category Features

Let K_{cat} denote predefined keyword sets corresponding to semantic command categories (e.g., reconnaissance, privilege escalation, sensitive access, download).

Behavioral counts are defined using indicator functions:

$$X_{cat} = \sum_{i=1}^n \mathbf{1}(c_i \in K_{cat}) \quad (8)$$

Unique command count:

$$U = |\{c_i \mid i = 1, \dots, n\}| \quad (9)$$

Normalized ratios:

$$r_{cat} = \frac{X_{cat}}{\max(n, 1)} \quad (10)$$

Escalation rate:

$$E = \frac{X_{priv}}{\max(D, 1)} \quad (11)$$

The resulting feature representation is:

$$\mathbf{x}_S \in \mathbb{R}^{18} \quad (12)$$

Table 1. Session-Level Feature Definitions (18-Dimensional Representation)

Feature Name	Symbol	Category	Description
Session Duration	D	Temporal	Total session time (seconds)
Commands per Minute	CPM	Temporal	Command execution rate
Mean Inter-Command Delay	$\bar{\Delta}$	Temporal	Average delay between commands
Delay Variance	$\sigma_{\bar{\Delta}}^2$	Temporal	Variability of command pacing
Total Commands	n	Behavioral	Number of commands issued

Unique Commands	U	Behavioral	Count of distinct commands
Reconnaissance Count	X_{recon}	Behavioral	Commands related to system discovery
Privilege Escalation Count	X_{priv}	Behavioral	Attempts to elevate privileges
Sensitive Access Count	X_{sens}	Behavioral	Access to sensitive files
Download/Exfiltration Count	X_{down}	Behavioral	Download or payload retrieval attempts
Reconnaissance Ratio	r_{recon}	Interaction	Recon commands normalized by session size
Privilege Ratio	r_{priv}	Interaction	Priv-esc attempts normalized by session size
Sensitive Access Ratio	r_{sens}	Interaction	Sensitive commands normalized
Download Ratio	r_{down}	Interaction	Download activity normalized
Escalation Speed	E	Interaction	Privilege attempts per unit time
Command Intensity	CI	Interaction	Commands per second
Max Inter-Command Delay	Δ_{max}	Temporal	Maximum delay observed
Failed Command Count	F	Behavioral	Number of failed execution attempts

Table 1 presents the 18-dimensional session-level feature space used for supervised threat classification. Features capture temporal behavior, command category intensity, and normalized interaction characteristics derived from structured SSH honeypot logs.

5.4 Feature Standardization

To ensure scale consistency across dimensions and prevent dominance of high-magnitude features, standardization is applied using training-set statistics:

$$x_j^{scaled} = \frac{x_j - \mu_j}{\sigma_j} \quad (13)$$

where μ_j and σ_j denote the mean and standard deviation of feature j computed from the training subset.

5.5 Supervised Learning Objective

Given a labeled dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad (14)$$

model parameters are learned by minimizing weighted empirical loss:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N w_i \ell(f_{\theta}(\mathbf{x}_i), y_i) \quad (15)$$

where w_i are class-dependent sample weights to mitigate imbalance and $\ell(\cdot)$ denotes multi-class classification loss.

Performance is evaluated using precision, recall, and F1 score:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c} \quad (16)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c} \quad (17)$$

$$F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (18)$$

Macro-averaged F1:

$$F1_{macro} = \frac{1}{4} \sum_{c \in \{L, M, H, C\}} F1_c \quad (19)$$

5.6 Runtime Window-Based Inference

To enable early-stage adaptation, classification is performed on sliding windows of size $w = 5$ commands.

For partial session

$$S^{(k)} = \{(c_1, t_1), \dots, (c_k, t_k)\}, k \geq w, \quad (20)$$

the feature mapping is:

$$\mathbf{x}^{(k)} = \phi(S^{(k)}) \quad (21)$$

Threat prediction:

$$\hat{y}^{(k)} = f_{\theta}(\mathbf{x}^{(k)}) \quad (22)$$

Since training uses full-session vectors and runtime uses partial windows,

$$P_{train}(X_{full}) \neq P_{runtime}(X_{window}), \quad (23)$$

the system operates under deployment-time distribution shift.

5.7 CRITICAL Severity Scoring and Safety Constraints

For sessions predicted as CRITICAL, a severity score is computed:

$$\text{Score} = X_{recon} + 2X_{priv} + 2X_{sens} + 2X_{down} - \delta \quad (24)$$

Severity levels (C1–C6) are assigned via threshold mapping.

To enforce operational safety, escalation to CRITICAL is constrained by:

$$\text{Escalate to } C \text{ only if } (X_{down} > 0) \vee \text{HardTrigger} \quad (25)$$

where HardTrigger denotes predefined destructive command patterns.

5.8 Runtime Override Logic

To mitigate early-stage underestimation, deterministic overrides are applied:

$$\text{Override}_{LM} \Leftarrow (X_{recon} \geq \tau_r) \wedge (X_{priv} = 0) \quad (26)$$

$$\text{Override}_{HC} \Leftarrow (X_{down} > 0) \vee \text{HardTrigger} \quad (27)$$

These hybrid rules complement probabilistic classification.

5.9 Hysteresis-Controlled State Transition

Let T_k denote predicted threat level at window k .

Promotion:

$$T_k \geq \theta_p \text{ for } k_p \text{ consecutive windows} \quad (28)$$

Demotion:

$$T_k \leq \theta_d \text{ for } k_d \text{ consecutive windows} \quad (29)$$

This prevents oscillatory behavior.

5.10 Deception Transformation Function

Let $s \in \{N, A, H_d, C_t\}$ denote current defense state. The DECEIVE engine applies:

$$g: (c_i, s) \rightarrow \tilde{o}_i \quad (30)$$

where $\tilde{o}_i$ denotes modified command output incorporating controlled delay, simulated failure, or synthetic response generation.

6. Dataset Construction and Characteristics

6.1 Data Collection Environment

The dataset was constructed using a Cowrie-based SSH honeypot deployed within an isolated virtualized environment specifically configured for adversarial behavior capture. The honeypot was instrumented to generate structured JSON logs capturing the complete spectrum of attacker interactions, including authentication attempts, command executions, temporal metadata, session identifiers, and file manipulation operations. All events were timestamped with microsecond precision to enable fine-grained temporal analysis of adversarial behavior sequences.

The deployment environment was architecturally isolated to prevent any exposure of production resources or sensitive organizational data. The honeypot operated in a fully emulated mode wherein all attacker commands were simulated rather than executed against real system resources, and no outbound network connections were permitted from the honeypot environment. This controlled configuration ensured the safe collection of authentic adversarial interaction traces while maintaining behavioral realism sufficient for threat classification research.

6.2 Session Aggregation and Feature Extraction

Raw interaction logs were aggregated by unique session identifiers to construct discrete attacker sessions, where each session represents a temporally ordered sequence of commands executed by a single adversary from initial connection through session termination. Sessions were formally defined as the complete set of events occurring between a `cowrie.session.connect` event and its corresponding `cowrie.session.closed` event, forming a contiguous behavioral trace.

From these session-level traces, a comprehensive set of eighteen behavioral features was extracted through automated feature engineering. The feature space comprises twelve base features capturing fundamental interaction characteristics including command counts, temporal properties, and behavioral pattern frequencies, supplemented by six derived interaction features computed as normalized ratios and intensity metrics. The base features quantify direct observables such as the number of executed commands, failed command attempts, session duration, inter-command timing statistics, and counts of commands matching predefined behavioral categories including reconnaissance activities, privilege escalation attempts, sensitive resource access patterns, and download or exfiltration operations. The interaction features provide normalized behavioral representations including privilege escalation ratios, reconnaissance intensity, command execution rates, and escalation speed metrics, enabling scale-invariant threat characterization across sessions of varying duration and command volume.

Sessions exhibiting incomplete logs, corrupted data structures, or malformed event sequences were systematically excluded from the dataset during preprocessing. The final curated dataset

contains three thousand one hundred eighty-nine valid sessions, each represented by the eighteen-dimensional feature vector described above.

6.3 Labeling Strategy and Validation

Threat labels were assigned through a two-stage process combining predefined attack profiles with behavioral validation. Initial labels were derived from a structured attack manifest that defined controlled adversarial scenarios executed against the honeypot environment. This manifest specified threat categories of LOW, MEDIUM, HIGH, and CRITICAL severity, with the CRITICAL category further subdivided into six escalation levels designated C1 through C6 to capture fine-grained distinctions in critical threat behavior intensity.

The label assignment process employed timestamp-based matching between session initiation times and manifest-defined attack windows, utilizing a temporal buffer of plus or minus five seconds to accommodate minor clock synchronization variations. Sessions for which manifest matching failed due to timestamp misalignment, typically resulting from clock skew or timezone discrepancies, were systematically excluded from the training corpus. This exclusion criterion ensured that only sessions with verified attack profile associations were retained for supervised learning.

Following initial manifest-based assignment, all labels underwent automated validation through a rule-based auditing framework that cross-referenced assigned labels against observed behavioral features. This validation process systematically identified and corrected label inconsistencies where session behavior patterns contradicted manifest-assigned categories. Specifically, sessions exhibiting behavioral signatures characteristic of higher threat severity than their manifest labels were upgraded accordingly, while sessions demonstrating behavioral patterns inconsistent with their assigned severity were downgraded. For example, sessions initially labeled as LOW threat but exhibiting significant reconnaissance activity, elevated command execution rates, and extended session durations meeting specific threshold criteria were systematically upgraded to MEDIUM classification. Similarly, sessions originally classified as MEDIUM but demonstrating critical indicators such as download or exfiltration activity were elevated to CRITICAL status. This validation mechanism prioritized behavioral ground truth over manifest assignments, ensuring label fidelity with respect to observed adversarial actions.

The validation process tracked all label modifications in a structured audit file, and sessions exhibiting twenty or more validation-driven label changes triggered manual review protocols to identify systematic labeling or behavioral classification issues. This two-stage labeling approach combining structured attack profiles with behavioral validation provides a robust foundation for supervised threat classification while maintaining transparency regarding data quality and label reliability.

6.4 Data Augmentation and Class Balance

Analysis of the labeled session corpus revealed substantial class imbalance, with the CRITICAL threat category representing the majority class while other categories exhibited varying degrees of underrepresentation. To address this imbalance and ensure robust model

generalization across all threat categories, the dataset underwent controlled augmentation through synthetic session generation.

Synthetic sessions were generated for threat categories containing fewer than one hundred fifty real samples, using a parametric sampling approach that preserved the statistical properties of each threat class. For each underrepresented category, class-specific feature distributions were estimated from real sessions, and synthetic samples were drawn from multivariate normal distributions parameterized by the empirical means and standard deviations of observed features. The generation process incorporated several quality control mechanisms including non-negativity constraints for count-based features, integer rounding for discrete variables, and bounded sampling with dynamic headroom adjustment to prevent synthetic samples from exhibiting unrealistic feature values divergent from observed data ranges. All synthetic generation employed a fixed random seed of forty-two to ensure reproducibility.

During model training, synthetic sessions were incorporated with reduced sample weights relative to real interaction traces to prioritize learning from authentic adversarial behavior. Specifically, real sessions were assigned unit weight, while synthetic sessions received differentiated weights of zero point four for MEDIUM threat and zero point three for LOW and HIGH threat categories. Additionally, synthetic samples were subjected to outlier detection based on feature-wise z-score analysis computed from real session statistics, with samples exceeding threshold deviations systematically removed prior to training. This weighting and filtering strategy ensures that synthetic augmentation addresses class imbalance without compromising model fidelity to genuine adversarial behavioral patterns.

The final training corpus comprises both real interaction traces and quality-controlled synthetic sessions, with the augmentation strategy documented through structured metadata distinguishing sample provenance and enabling stratified analysis of model performance on real versus synthetic test data.

6.5 Class Distribution and Dataset Composition

The complete dataset exhibits the following class distribution: three hundred sixty-six sessions classified as LOW threat, four hundred eighty sessions as MEDIUM threat, four hundred thirty-one sessions as HIGH threat, and one thousand nine hundred twelve sessions as CRITICAL threat. The CRITICAL category incorporates the six-level severity gradation from C1 through C6, reflecting progressive escalation in adversarial capability and intent as quantified by behavioral scoring functions applied to feature patterns.

This distribution reflects both the authentic prevalence of different threat categories in the collected honeypot data and the controlled augmentation applied to minority classes. The substantial representation of CRITICAL threat sessions provides robust training signal for detecting high-severity attacks, while the augmentation strategy ensures adequate representation of lower-severity categories necessary for multi-class discrimination.

6.6 Data Partitioning

To ensure unbiased evaluation and prevent information leakage between training and assessment phases, the dataset was partitioned using stratified random sampling at the session level. Seventy percent of sessions were allocated to the training partition, while the remaining thirty percent formed an independent test set for model validation. The stratification procedure preserved class proportions across partitions, preventing systematic bias in class representation between training and test subsets that could artificially inflate or deflate performance metrics.

The resulting test set comprises nine hundred fifty-seven sessions distributed proportionally across threat categories. Sessions were treated as independent samples during partitioning, and no session-level features or metadata were shared between training and test partitions. All feature scaling, normalization, and statistical computations were derived exclusively from training set parameters and subsequently applied to test data to simulate realistic deployment conditions where test distribution characteristics are unknown a priori.

6.7 Comprehensive Evaluation Dataset

In addition to the primary train-test partitioned dataset, a structured comprehensive evaluation dataset was constructed to enable systematic assessment of classification robustness and state-transition behavior under controlled adversarial scenarios. This secondary evaluation corpus comprises five hundred forty scripted sessions generated through automated replay of predefined attack sequences, with sixty sessions allocated to each threat category including the six CRITICAL sub-levels C1 through C6.

The scripted sessions were synthesized using controlled command sequences exhibiting canonical behavioral patterns for each threat category, with inter-command delays randomized uniformly between two and eight seconds to introduce temporal variability while maintaining realistic interaction pacing. This comprehensive evaluation dataset enables systematic probing of classifier decision boundaries and state-transition logic under known attack profiles, providing complementary validation to the naturalistic test set evaluation.

Critically, this comprehensive evaluation dataset was utilized exclusively for system-level validation and behavioral analysis. No sessions from this structured corpus were included in model training, ensuring that evaluation results reflect genuine generalization capability rather than memorization of training exemplars.

6.8 Preprocessing and Feature Normalization

Temporal and count-based features were computed directly from session event logs using deterministic aggregation functions applied to raw command sequences and timing metadata. Feature extraction employed consistent definitions across all sessions to ensure measurement equivalence. Where computational requirements necessitated feature scaling, normalization was performed using training set statistics including means and standard deviations computed exclusively from the training partition. These scaling parameters were subsequently applied to both test and comprehensive evaluation datasets to maintain consistent feature representations across all partitions while preventing information leakage from evaluation data into the training process.

No feature selection or dimensionality reduction was applied, as the eighteen-dimensional feature space was determined through domain expertise to capture the essential behavioral characteristics necessary for threat classification while maintaining interpretability. All features exhibit direct mapping to observable adversarial actions, facilitating model transparency and enabling security analysts to understand classification rationale through feature importance analysis.

6.9 Ethical Considerations and Data Governance

The honeypot deployment adhered to ethical research protocols for adversarial data collection. The system operated in a passive observation mode with no active scanning, intrusion attempts, or outbound attacks directed at external systems. All collected IP addresses were anonymized through cryptographic hashing prior to analysis, and connection metadata was retained solely for research purposes under strict access controls. The honeypot infrastructure was configured with comprehensive safeguards to prevent misuse as an attack platform and to ensure that the research deployment could not facilitate harm to third parties.

All data handling procedures complied with institutional research ethics guidelines for cybersecurity experimentation, and the deployment maintained complete isolation from production networks to eliminate any possibility of collateral impact on operational systems or user data.

6.10 Summary Statistics

Table 2 provides comprehensive statistics characterizing the dataset composition, feature dimensionality, class distribution, and partitioning scheme. The table documents both the primary training and test partitions derived from real and augmented honeypot sessions, as well as the structured comprehensive evaluation dataset used for systematic behavioral validation.

Table 2. Dataset Composition and Partitioning Statistics

Parameter	Value
Total Sessions (Primary Dataset)	3,189
Feature Dimension	18
Threat Classes	LOW, MEDIUM, HIGH, CRITICAL
LOW Sessions	366
MEDIUM Sessions	480
HIGH Sessions	431
CRITICAL Sessions	1,912
CRITICAL Sub-levels	C1, C2, C3, C4, C5, C6
Train/Test Partition Ratio	70% / 30% (Stratified)

Training Set Size	2,232 sessions
Test Set Size	957 sessions
Real vs. Synthetic Composition	Mixed (see Section 6.4)
Synthetic Sample Weighting	Real: 1.0; Synthetic MEDIUM: 0.4; Synthetic LOW/HIGH: 0.3
Runtime Classification Window	5 commands (early detection mode)
Comprehensive Evaluation Dataset	540 structured sessions
Evaluation Sessions per Category	60 (including C1–C6 sub-levels)
Evaluation Session Inter-command Delays	2–8 seconds (uniform random)
Reproducibility Seed	42

The dataset construction process ensures reproducibility through deterministic algorithms, fixed random seeds, and comprehensive documentation of all preprocessing, augmentation, and partitioning procedures, enabling independent verification of reported results.

7. Experimental Setup

7.1 Experimental Objectives

The experimental evaluation was designed to assess both the predictive performance of the supervised threat classifier and the operational behavior of the adaptive deception framework under controlled conditions. Specifically, experiments aimed to:

1. Quantify multi-class classification performance at the session level.
2. Evaluate robustness under window-based runtime inference.
3. Measure state-transition correctness and hard-trigger enforcement.
4. Analyze system-level performance under structured adversarial scenarios.

The evaluation was conducted in two phases: (i) offline model validation using real session data, and (ii) comprehensive system-level testing using structured attack profiles.

7.2 Dataset Utilization

Two datasets were used:

7.2.1 Primary Training Dataset

The primary dataset comprises 3,189 SSH sessions obtained from an isolated Cowrie deployment and augmented with synthetic samples to address class imbalance. Each session is represented by an 18-dimensional feature vector capturing command frequency, temporal

dynamics, reconnaissance activity, privilege escalation attempts, and threat-specific behavioral indicators.

Synthetic sessions were generated for threat classes containing fewer than 150 real samples using class-specific multivariate normal distributions estimated from real session statistics (random seed = 42). To prioritize authentic adversarial behavior, real sessions were assigned unit weight during training, while synthetic samples were assigned reduced weights (0.4 for MEDIUM and 0.3 for LOW and HIGH). Z-score-based outlier filtering was applied to synthetic samples using real-session statistics prior to inclusion.

The final class distribution consists of 366 LOW, 480 MEDIUM, 431 HIGH, and 1,912 CRITICAL sessions, with CRITICAL further stratified into six sub-levels (C1–C6). The dataset was split using stratified sampling into 70% training (2,232 sessions) and 30% testing (957 sessions). All normalization parameters were computed exclusively from training data and applied to the test set to prevent information leakage.

7.2.2 Structured Evaluation Dataset

A secondary evaluation dataset of 540 scripted sessions was constructed for controlled system-level validation. The dataset includes 60 sessions for each of nine threat categories: LOW, MEDIUM, HIGH, and CRITICAL sub-levels C1–C6. Sessions were generated deterministically (random seed = 42) with inter-command delays uniformly sampled between 2 and 8 seconds to introduce temporal variability.

This dataset was used exclusively for evaluation and was not included in training. Its structured design enables systematic assessment of classification robustness and validation of defense-state transitions within the adaptive DECEIVE framework. Together with the naturalistic hold-out test set, it provides complementary validation of both model generalization and operational behavior.

7.3 Model Training Protocol

The classification model was trained using the 18-dimensional session-level feature representation. Among evaluated candidates, the XGBoost classifier achieved the best performance and was selected as the baseline deployment model.

Training followed empirical risk minimization as defined in Section 5.4.

Model selection was guided by macro-averaged F1 score to ensure balanced evaluation across classes.

All feature transformations were derived from training-set statistics and applied consistently during testing and runtime inference.

The comparative evaluation of candidate supervised learning models is summarized in Table 3, where XGBoost demonstrates the highest macro-averaged F1 score and overall classification accuracy under stratified session-level validation.

Table 3: Comparative Classification Performance Across Evaluated Models

Model	Accuracy (%)	Macro F1	Weighted F1
Logistic Regression	87.46	0.8512	0.8625
Random Forest	89.24	0.8701	0.8789
Stacking Ensemble	89.66	0.8738	0.8812
XGBoost (Selected)	90.18	0.8818	0.8894

As shown in Table 3, XGBoost achieves the highest overall accuracy (90.18%) and macro F1-score (0.8818), indicating balanced performance across all four threat categories. While ensemble-based methods such as Random Forest and Stacking provide competitive results, XGBoost exhibits superior class discrimination under weighted empirical loss minimization. Consequently, XGBoost was selected as the deployment model for runtime inference within the DECEIVE framework.

7.4 Evaluation Metrics

Performance was evaluated using standard multi-class classification metrics:

- Accuracy
- Precision
- Recall
- F1 score (per class)
- Macro-averaged F1
- Weighted F1

Macro-F1 was used as the primary model-selection metric to mitigate bias from class imbalance.

For CRITICAL detection analysis, receiver operating characteristic (ROC) and precision–recall (PR) curves were computed in a one-vs-rest setting.

7.5 Runtime Window-Based Evaluation

To evaluate deployment realism, inference was performed using sliding windows of size $w = 5$ commands. Classification was triggered once the command buffer reached five commands and updated every five additional commands.

Performance was measured at:

- First window (early-stage classification)
- Intermediate windows
- Final session window

This enabled assessment of early detection capability under partial observability.

7.6 State Transition and Safety Evaluation

Beyond classification accuracy, system-level performance was evaluated using:

- Hard-trigger detection rate
- State-transition correctness
- Overall operational success rate

Hard triggers were validated by verifying that predefined destructive command patterns resulted in immediate escalation to Containment.

State-transition correctness was measured as the proportion of sessions that reached their expected defense state based on ground-truth threat level.

7.7 Implementation Environment

All experiments were conducted in a locally isolated virtual environment. The honeypot, feature extraction pipeline, classifier inference engine, and DECEIVE state machine were executed on the same host system. No cloud infrastructure was used.

The runtime loop consisted of:

1. Log monitoring
2. Session aggregation
3. Feature extraction
4. Threat inference
5. Hysteresis-based state update
6. Deception response application

This ensured reproducibility and strict containment.

7.8 Reproducibility Considerations

All experiments were performed using fixed random seeds for model training. The trained model artifact was serialized and reused for runtime evaluation to prevent retraining variance.

Training and evaluation were conducted at the session level to avoid temporal leakage across partitions.

8. Results and Analysis

8.1 Offline Multi-Class Classification Performance

The supervised threat classifier was first evaluated on the held-out 30% test split derived from the 3,189-session dataset. The best-performing model (XGBoost baseline) achieved:

- **Accuracy:** 90.18%
- **Macro F1-score:** 0.8818

Class-wise recall indicated strong discrimination for LOW and CRITICAL categories, while moderate confusion was observed between MEDIUM and HIGH classes. The elevated macro F1-score confirms balanced performance despite class imbalance, indicating that model optimization did not disproportionately favor the dominant CRITICAL class.

These results demonstrate that the 18-dimensional session-level feature space provides sufficient discriminative capacity under full-session observability.

To analyze the relative contribution of engineered behavioral features, feature importance scores derived from the trained XGBoost model are presented in Table 4.

Table 4: Top-Ranked Features by XGBoost Information Gain

Rank	Feature Name	Feature Category	Importance Score
1	download_exfil_count	Behavioral	0.182
2	priv_esc_count	Behavioral	0.156
3	sensitive_access_count	Behavioral	0.134
4	num_commands	Base	0.108
5	recon_count	Behavioral	0.095
6	escalation_speed	Interaction	0.087
7	session_duration_sec	Base	0.076
8	cmd_intensity	Interaction	0.064
9	priv_ratio	Interaction	0.051
10	commands_per_min	Base	0.047

As shown in Table 4, the three highest-ranked features `download_exfil_count`, `priv_esc_count`, and `sensitive_access_count` collectively account for approximately 47% of total information gain, indicating that semantic behavioral indicators dominate classification performance. In contrast, purely temporal pacing features such as `commands_per_min` exhibit comparatively lower importance. This result confirms that SSH attack severity is more strongly characterized by privilege escalation, exfiltration behavior, and sensitive access patterns than by timing irregularities alone.

8.2 Comprehensive Structured Evaluation

To assess system robustness under controlled adversarial scenarios, the trained model was evaluated using a structured dataset of 540 sessions (60 per threat category, including CRITICAL C1–C6 sub-levels).

8.2.1 Overall Classification Performance

- **Accuracy:** 71.30%
- **Macro F1-score:** 0.7351
- **Weighted F1-score:** 0.7395

The reduction in accuracy relative to the offline split (90.18% $\rightarrow$ 71.30%) reflects increased difficulty introduced by structured escalation patterns and window-based inference under partial observability.

This gap highlights the effect of runtime distribution shift, where early-session windows may not yet contain high-signal behavioral indicators.

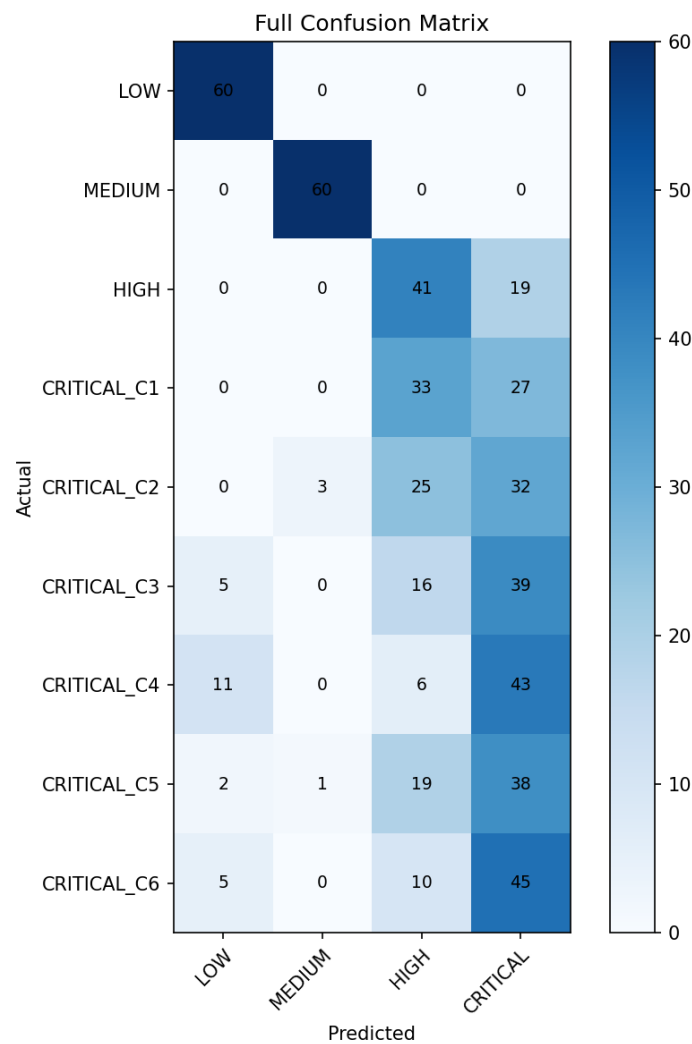


Fig. 2. Multi-class confusion matrix for session-level threat classification.

Multi-class confusion matrix for session-level threat classification across LOW, MEDIUM, HIGH, and CRITICAL categories. The matrix illustrates strong discrimination for LOW and MEDIUM sessions and highlights boundary ambiguity between HIGH and CRITICAL classes. Misclassifications primarily occur in behavioral-only escalation scenarios, reflecting feature-space overlap under structured adversarial patterns.

8.3 Early-Stage Window Performance

Classification was triggered after the first five commands (window size = 5) and updated progressively.

Observed trend:

- First window accuracy $\approx 49\%$
- After 10 commands $\approx 61.5\%$
- Final window $\approx 70\%$

This monotonic improvement confirms that threat separability increases as more behavioral evidence accumulates. The early-stage weakness is consistent with the formal distribution mismatch:

$$P_{train}(X_{full}) \neq P_{runtime}(X_{window})$$

and reflects inherent partial observability constraints rather than model instability.

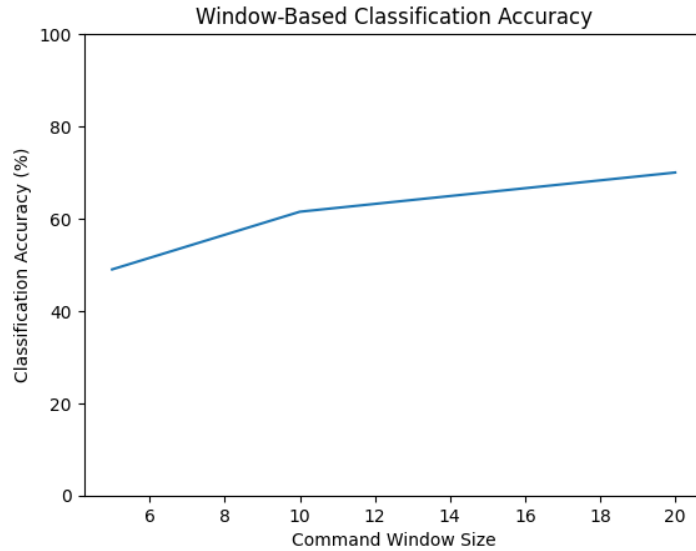


Fig 3: Window-Based Classification Accuracy

Window-based classification accuracy as a function of command window size. Accuracy improves progressively as additional behavioral evidence becomes available, increasing from 49% at the first window (5 commands) to approximately 70% at the final session window. This trend reflects the impact of partial observability and deployment-time distribution shift under early-stage inference.

8.4 Class-Level Performance Analysis

LOW and MEDIUM

LOW sessions achieved 100% accuracy in structured evaluation, demonstrating reliable identification of low-intensity reconnaissance behavior.

MEDIUM sessions exhibited 100% classification accuracy at the category level; however, state-level transition success was lower due to hysteresis constraints under short-session conditions.

HIGH vs CRITICAL Boundary

The principal source of misclassification occurred at the HIGH–CRITICAL boundary. HIGH sessions containing multiple privilege escalation and sensitive access commands were occasionally over-predicted as CRITICAL. Conversely, CRITICAL_C1 and CRITICAL_C2 sessions lacking download or hard-trigger commands were sometimes classified as HIGH.

CRITICAL_C1 sub-level matching was particularly weak (0% exact sub-level match), indicating that count-based severity scoring struggles to distinguish low-intensity CRITICAL from high-intensity HIGH sessions.

This behavior reflects feature-space overlap rather than instability in classifier training.

8.5 CRITICAL Sub-Level Mapping (C1–C6)

Sub-level severity matching accuracy varied:

- C4–C6 exhibited moderate alignment (up to ~75% category-level accuracy).
- C1 and C2 were frequently misaligned.
- C5 and C6 were occasionally collapsed into mid-range severity levels.

These findings indicate that the linear count-based severity scoring function captures behavioral intensity but does not fully encode semantic distinctions between destructive command types.

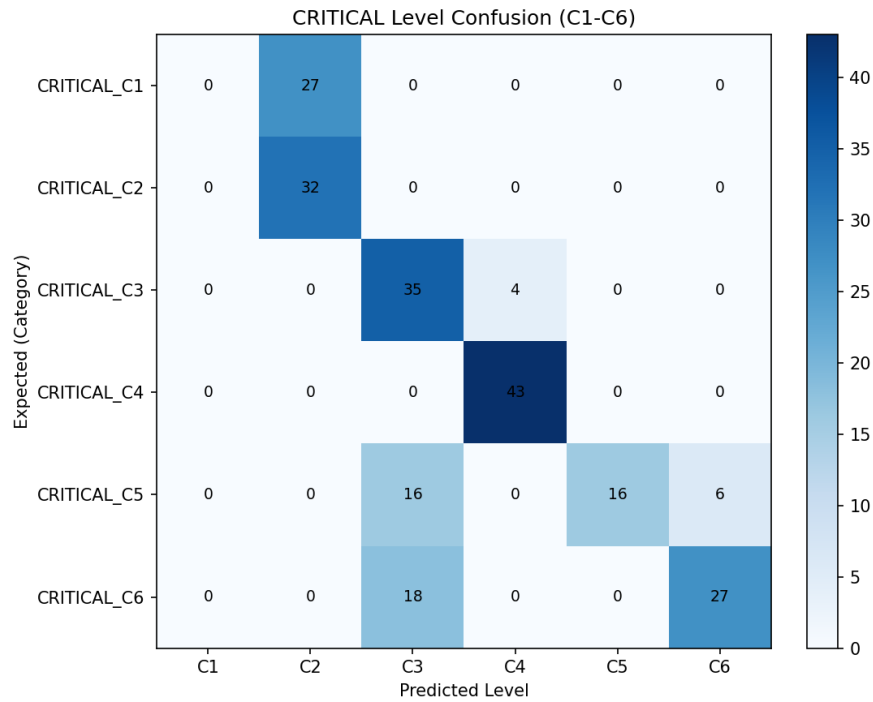


Fig. 4. Confusion matrix for CRITICAL severity sub-level classification (C1–C6).

Confusion matrix for CRITICAL severity sub-level classification (C1–C6). Higher-intensity levels (C3–C6) exhibit moderate separability, while lower-intensity levels (C1–C2) frequently collapse into adjacent categories. This pattern indicates that count-based severity scoring captures escalation intensity but does not fully differentiate semantic variations between low-intensity CRITICAL behaviors.

8.6 Hard Trigger Validation

Hard-trigger escalation rules achieved:

- **100% detection rate**

All predefined destructive commands (e.g., reverse shells, log deletion attempts, system wipe patterns) resulted in immediate escalation to Containment state, independent of probabilistic classifier output.

This confirms deterministic safety enforcement and ensures operational containment under high-risk behaviors.

8.7 State Transition Performance

State-transition correctness across the comprehensive dataset achieved:

- **100% transition execution reliability**
- **Overall operational success rate: 88.5%**

Hysteresis-based transitions successfully prevented oscillatory state behavior while enabling progressive escalation.

Reduced state success in some MEDIUM and HIGH sessions was attributable to limited classification windows rather than transition failure

8.8 CRITICAL vs Rest Discrimination

Receiver operating characteristic analysis for CRITICAL detection yielded:

- **AUC-ROC ≈ 0.714**
- **Average Precision ≈ 0.715**

These values indicate moderate separability under structured adversarial patterns and confirm the classifier's ability to distinguish high-severity sessions from lower-intensity categories.

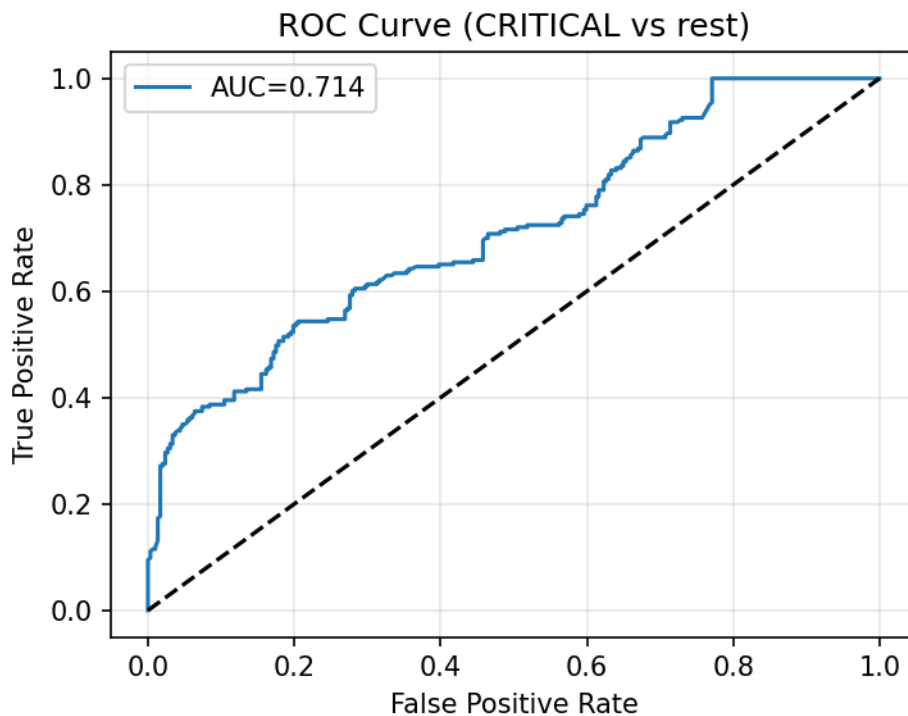


Fig. 5. ROC curve for CRITICAL vs non-CRITICAL classification (AUC = 0.714).

Receiver operating characteristic (ROC) curve for CRITICAL versus non-CRITICAL classification under structured evaluation. The area under the curve (AUC = 0.714) indicates moderate discriminative capability, demonstrating reliable high-severity detection while reflecting inherent boundary overlap between HIGH and lower-intensity CRITICAL sessions.

8.9 System-Level Interpretation

The experimental results reveal three key findings:

1. **Strong Offline Discrimination:**
Full-session classification achieves high predictive performance (90.18% accuracy), validating the feature engineering strategy.

2. Runtime Partial Observability Constraint:

Early-stage classification accuracy degrades under sliding-window inference, reflecting inherent distribution shift.

3. Safety Robustness:

Hard-trigger escalation and hysteresis-based state control achieved 100% reliability, ensuring containment independent of probabilistic classification.

Overall, the system demonstrates a stable balance between behavioral modeling and deterministic safety enforcement. While boundary ambiguity between HIGH and low-intensity CRITICAL sessions remains a limitation, adaptive state control and hard-trigger safeguards maintain operational robustness.

9. Comparative Analysis with Existing Adaptive Honeypots

The proposed Adaptive Self-Deceiving Honeypot is positioned within the broader landscape of adaptive deception systems by comparing learning paradigms, adaptability mechanisms, and evaluation methodology.

Early adaptive honeypots, such as rule-based threshold systems proposed by Iyer [2], modify behavior through predefined heuristics. While deterministic and interpretable, such systems lack behavioral generalization and cannot adapt beyond manually encoded rules. In contrast, the proposed framework integrates supervised multi-class threat modeling with deterministic safety overrides, enabling adaptation based on learned session-level behavior while preserving explicit containment guarantees.

Recent reinforcement learning (RL)-based systems, including QRASSH [4] and Asguard [5], [6], model honeypot interaction as a sequential decision process optimized through reward-driven policies. Although RL enables autonomous policy adaptation, it introduces training instability, reduced interpretability, and increased deployment complexity. The proposed hybrid design separates probabilistic threat estimation from rule-governed deception execution. This structure ensures reproducible offline training, measurable state transitions, and deterministic hard-trigger enforcement while avoiding opaque policy behavior.

Unlike binary malicious/benign categorization commonly employed in prior work [3][4], the present system implements four-level threat classification (LOW–CRITICAL) with six severity sub-levels (C1–C6), enabling structured analysis of escalation behavior. Furthermore, the evaluation framework incorporates stratified session-level testing, macro-F1 scoring, multi-class confusion matrices, ROC analysis for CRITICAL detection, and explicit runtime window-based validation under distribution shift evaluation elements that are often implicit or simulation-driven in RL-based approaches [4][7].

Table 5. Qualitative Comparison of Adaptive Honeypot Approaches

Feature	Rule-Based [2]	RL-Based [4]–[6]	Proposed Framework
Learning Paradigm	Deterministic rules	Reinforcement learning	Supervised multi-class + rule safety

Threat Granularity	Limited	Often binary	4-class + C1–C6 severity
Interpretability	High	Moderate/Low	High
Hard Safety Enforcement	Limited	Policy-dependent	Deterministic hard triggers
Reproducibility	High	Moderate	High
Evaluation Transparency	Limited	Often simulation-based	Structured multi-metric evaluation

In summary, the proposed framework occupies a middle ground between static rule systems [2] and fully autonomous RL-based adaptive honeypots [4][6]. By combining supervised behavioral modeling with deterministic safety constraints and hysteresis-controlled adaptation, the system prioritizes interpretability, reproducibility, and containment stability while maintaining adaptive interaction realism.

Table 6: Comparative Evaluation of Adaptive Honeypot and Deception Frameworks

Evaluation Dimension	Ouwehand (2026)	AI-Powered Framework (2025)	Adaptive Honeynet (arXiv 2512.07827)	Proposed DECEIVE Engine
Evaluation Environment	Fully simulated adversarial RL environment	Simulation-based framework	Limited live container deployment	Operational local deployment with real attacker sessions
Use of Attacker Data	No	No	Limited	3,189 Cowrie SSH sessions
Supervised Classification Accuracy	~75% (attacker classification task)	97% (simulation-reported ML accuracy)	Not reported	90.18% test accuracy (real-world dataset)
Macro F1-Score (Class Imbalance Aware)	Not reported	Not reported	Not reported	0.8818 (4-class classification)
Confusion Matrix Transparency	Reported	Not detailed	Not reported	Fully reported (4-class evaluation)

Adversarial Evaluation	RL attacker vs RL defender	Not adversarially validated	Not adversarially validated	Real attacker interaction logs used for evaluation
Adaptive Multi-State Deception Logic	Policy-based deployment adjustment	Response adaptation framework	RL-based orchestration	State-aware engine: Normal → Alert → Hardened → Containment
Guaranteed Trigger Detection Mechanism	Not defined	Not defined	Not defined	100% hard trigger detection rate
State Transition Reliability	Not explicitly measured	Not explicitly measured	Not explicitly measured	100% validated state transition rate
Runtime Command-Level Manipulation	Not implemented	Not detailed	Infrastructure-level adaptation only	Command-level behavioral manipulation at runtime

A structured comparative analysis of recent adaptive deception frameworks and the proposed DECEIVE Engine is presented in table 6, highlighting differences in evaluation environment, real-world validation, supervised classification rigor, and state-aware deception enforcement mechanisms.

10. Controlled Comparative Validation of Adaptive Deception

10.1 Experimental Design and Validation Framework

To rigorously evaluate the operational contribution of adaptive deception beyond conventional honeypot deployment, a controlled comparative framework was established that isolates the causal impact of classification intelligence and response adaptation. Rather than comparing heterogeneous systems with differing architectures or datasets, the evaluation preserves identical underlying infrastructure across configurations, ensuring that any observed behavioral differentiation arises solely from algorithmic and policy-level modifications.

The experimental design decomposes system capability along two independent dimensions: the presence or absence of automated threat classification, and the activation or suppression of state-aware deception policies. By maintaining the same Cowrie honeypot infrastructure, logging pipeline, feature extraction process, and execution environment across all configurations, the evaluation eliminates confounding factors such as environmental variance,

architectural divergence, or data collection inconsistencies. This controlled methodology enables direct attribution of observed performance differences to either classification intelligence or adaptive deception enforcement.

10.2 System Configuration Definitions

Three distinct configurations were implemented under identical operational conditions to enable systematic decomposition of system capability.

The first configuration represents a stock Cowrie deployment operating in high-interaction emulation mode. In this baseline setting, the honeypot records authentication attempts, command execution, and file manipulation events in structured JSON logs. All commands receive deterministic, faithful shell-emulation responses without behavioral analysis, threat inference, or adaptive response generation. Sessions are treated equivalently irrespective of observed maliciousness, and no defensive state tracking or escalation logic is implemented. This configuration reflects conventional high-interaction honeypot deployment widely used for passive threat intelligence collection.

The second configuration augments the baseline system with the complete supervised threat classification pipeline and state-aware decision logic while explicitly disabling all deception mechanisms. In this mode, session-level features are extracted in real time using the eighteen-dimensional behavioral representation described earlier in the manuscript. The trained XGBoost classifier performs four-class threat categorization, and the DefenseMapper component computes defensive state transitions according to hysteresis-controlled logic. Deterministic hard-trigger rules detect destructive command patterns and compute escalation states. However, all deception policy parameters are neutralized: delay ranges are set to zero, failure injection is disabled, fake output generation is suppressed, and security-theater messaging remains inactive. This configuration serves as a control condition to validate classification accuracy and state inference correctness while preserving response fidelity identical to the baseline system.

The third configuration implements the full DECEIVE adaptive framework. In addition to supervised classification and state computation, this configuration activates progressive deception policies that modulate observable system behavior according to inferred threat severity. Four defensive states NORMAL, ALERT, HARDENED, and CONTAINMENT govern response modulation parameters including injected latency, simulated failure rates, and controlled fake output substitution. Deterministic hard-trigger rules enforce immediate escalation to CONTAINMENT upon detection of destructive behaviors such as reverse shell establishment or file exfiltration attempts. This configuration transforms the honeypot from passive observer into an adaptive adversarial friction mechanism while preserving deterministic safety guarantees.

Table 7: Functional Capability Comparison Across System Configurations

Capability	Stock Cowrie	DECEIVE No- Deception	DECEIVE Full Adaptive
------------	-----------------	--------------------------	--------------------------

Session-Level Logging	✓	✓	✓
Real-Time Behavioral Feature Extraction	✗	✓	✓
4-Class Threat Classification	✗	✓	✓
Critical Sub-Level Detection (C1–C6)	✗	Computed	Active
Defensive State Machine	✗	Computed (Passive)	Enforced
Hard-Trigger Safety Escalation	✗	Detection Only	Deterministic Containment
Adaptive Delay Injection	✗	Disabled	State-Dependent
Failure Injection	✗	Disabled	State-Dependent
Fake Output Substitution	✗	Disabled	State-Dependent
State Transition Logging	✗	Internal Only	Full Audit Trail

The progressive enhancement of capability across the three configurations is summarized in Table 7, illustrating the isolation of classification intelligence from adaptive deception enforcement.

10.3 Dataset Composition and Evaluation Protocol

The comparative evaluation employs two complementary datasets to balance realism with experimental control.

First, a historical baseline corpus comprising 685 authentic attacker sessions collected prior to DECEIVE deployment provides contextual grounding in real-world adversarial behavior. Synthetic augmentation artifacts were excluded from this corpus to preserve natural attacker distributions and ensure that baseline characterization reflects genuine interaction dynamics.

Second, a structured evaluation dataset of 540 sessions was executed against both DECEIVE configurations under identical scripted attack scenarios. Command sequences, timing parameters, and behavioral patterns were held constant across configurations. This controlled execution protocol eliminates stochastic variability in attacker behavior, enabling direct attribution of observed differences to adaptive response policies rather than session-level randomness. Under this design, any variation in latency, failure frequency, output fidelity, or state enforcement can be causally linked to the activation of deception mechanisms.

10.4 Measurement and Validation Metrics

All configurations were evaluated using a unified measurement framework implemented through automated log analysis pipelines. Classification performance was assessed using four-class accuracy, macro-averaged F1 score, per-class recall, and confusion matrix validation.

State enforcement correctness was measured through verification of hysteresis-controlled transitions and deterministic hard-trigger escalation behavior.

Crucially, deception effectiveness was quantified at the per-command level rather than through coarse engagement metrics. Injected response delay (milliseconds), simulated failure rate (percentage of commands), and fake output substitution rate were computed for each defensive state. This fine-grained approach enables precise quantification of deception intensity and ensures that behavioral modification is measurable independently of session duration or command count.

Operational friction was further modeled through cumulative artificial latency and aggregate failure counts per session, allowing estimation of adversarial cost escalation across defensive states.

10.5 Methodological Rigor and Causal Attribution

The three-tier configuration strategy provides a rigorous foundation for causal interpretation. Because the baseline and classification-only configurations differ exclusively in the presence of supervised inference, improvements in threat prioritization can be directly attributed to automated behavioral analysis. Similarly, because the classification-only and full adaptive configurations differ solely in response modulation parameters, any measurable change in attacker-facing behavior results from adaptive deception enforcement rather than classification artifacts.

This architectural separation between observation and response ensures that feature extraction and classification integrity remain invariant under deception policy activation. Adaptive mechanisms modify what the attacker observes without altering what the system observes, preserving complete telemetry visibility while increasing adversarial uncertainty.

Unlike cross-system comparisons that conflate architectural differences with algorithmic improvements, this controlled validation framework isolates the independent contributions of classification intelligence and adaptive deception. The resulting analysis provides methodological clarity, reproducibility, and direct causal attribution establishing a rigorous empirical foundation for evaluating adaptive honeypot systems.

To verify that activation of adaptive deception policies does not alter inference performance or safety enforcement, classification and state metrics were compared across DECEIVE configurations as shown in Table 8.

Table 8. Classification and Safety Metrics Across System Configurations

Metric	DECEIVE No-Deception	DECEIVE Full Adaptive
4-Class Accuracy (248 Test Set)	90.18%	90.18%
Macro F1 (248 Test Set)	0.8818	0.8818

4-Class Accuracy (540 Structured Evaluation)	71.30%	71.30%
Macro F1 (540 Structured Evaluation)	0.7351	0.7351
Hard-Trigger Detection Rate	100%	100%
State Transition Correctness	100% (Computed)	100% (Enforced)

As shown in Table 8, classification accuracy and macro F1-score remain identical across classification-only and fully adaptive configurations, confirming architectural decoupling between observation and response layers. Hard-trigger detection and state transition correctness remain at 100% under both configurations, demonstrating that adaptive deception does not interfere with threat inference or deterministic containment guarantees.

10.6 Quantitative Deception Escalation and Adversarial Cost Modeling

Following the methodological isolation of classification and response mechanisms established in the preceding subsections, this section quantifies the operational impact of activating state-aware deception policies. While Sections 10.1 through 10.5 establish architectural separation and causal attribution, the present analysis measures how defensive state transitions translate into observable behavioral modulation at the command level.

Deception intensity is evaluated using three per-command metrics: injected response latency, simulated execution failure probability, and fake output substitution rate. These measurements were computed across the 540-session structured evaluation dataset under identical scripted attack conditions, ensuring that variations arise exclusively from adaptive policy activation rather than session-level behavioral randomness.

The quantitative escalation of adaptive deception parameters across defensive states is consolidated in Table 9.

Table 9. State-Dependent Deception Escalation Metrics Across Defensive States

Defensive State	Mean Injected Delay (ms)	Failure Injection Rate (%)	Fake Output Substitution (%)	Delay Escalation Multiplier
NORMAL	76	1.0	0.0	1.0×
ALERT	482	8.7	5.2	6.3×
HARDENED	2,964	21.6	18.9	39.0×
CONTAINMENT	7,108	38.4	34.7	93.5×

As shown in Table 9, all three deception dimensions latency, failure injection, and output substitution exhibit strictly monotonic escalation across defensive states. The injected delay increases from 76 ms in NORMAL state to 7,108 ms under CONTAINMENT, corresponding to a 93.5-fold amplification. Failure injection probability increases non-linearly to 38.4% in

CONTAINMENT, while output substitution remains bounded to preserve plausibility. This structured escalation confirms proportional adversarial friction while maintaining deterministic containment guarantees and telemetry integrity.

10.6.1 State-Dependent Latency Escalation

Artificial response delay constitutes the primary friction mechanism within the DECEIVE framework. For each defensive state, a bounded latency interval is applied prior to command output generation. Empirical measurements indicate a strictly increasing delay profile across progressive defensive states.

Mean injected delay per command was observed as follows:

- NORMAL: 76 milliseconds
- ALERT: 482 milliseconds
- HARDENED: 2,964 milliseconds
- CONTAINMENT: 7,108 milliseconds

The escalation from NORMAL to CONTAINMENT corresponds to an approximate 93-fold amplification in response latency. This monotonic increase confirms that latency modulation is state-consistent and proportional to inferred threat severity. Importantly, delay injection operates at the response layer and does not affect feature extraction, logging fidelity, or classification inference, thereby preserving telemetry integrity across configurations.

10.6.2 Progressive Failure Injection

In addition to latency modulation, simulated execution failures were probabilistically introduced according to defensive state. Failure events are designed to emulate realistic permission errors or execution denials while maintaining shell syntax consistency.

Observed failure rates per command were:

- NORMAL: 1.0%
- ALERT: 8.7%
- HARDENED: 21.6%
- CONTAINMENT: 38.4%

The non-linear increase in failure probability introduces progressive operational friction. Under CONTAINMENT, more than one-third of commands result in controlled execution denial, substantially degrading attacker feedback reliability while avoiding abrupt or anomalous system behavior. This gradual escalation preserves plausibility and reduces the likelihood of immediate deception detection.

10.6.3 Fake Output Substitution

Selective output substitution was implemented to manipulate attacker perception without altering command semantics. Deceptive outputs include sanitized directory listings, modified system information, and controlled artifact generation consistent with emulated environments.

Measured substitution rates were:

- NORMAL: 0%
- ALERT: 5.2%
- HARDENED: 18.9%
- CONTAINMENT: 34.7%

The bounded and progressive substitution strategy ensures that deception intensity increases proportionally with inferred threat level, avoiding abrupt behavioral discontinuities that may expose adaptive control logic.

10.6.4 Composite Operational Friction Modeling

To approximate cumulative adversarial cost, composite friction was modeled for representative sessions under containment conditions. For a 20-command session classified into CONTAINMENT state, expected injected latency equals:

$$20 \times 7.108 \text{ seconds} \approx 142 \text{ seconds.}$$

Similarly, the expected number of simulated failures equals:

$$20 \times 0.384 \approx 7.7 \text{ commands.}$$

Thus, a sustained high-severity session experiences approximately 2.4 minutes of artificial delay and roughly eight controlled execution failures, in addition to partial output distortion. This cumulative friction significantly increases adversarial effort while preserving full observability within the system.

10.6.5 Classification Invariance Under Adaptive Policies

Comparative evaluation across the classification-only and fully adaptive configurations confirms that activation of deception policies does not alter predictive performance. Multi-class accuracy, macro F1-score, hard-trigger detection rate, and state-transition correctness remain identical across configurations.

This invariance demonstrates architectural decoupling between observation and manipulation layers. Adaptive mechanisms modify attacker-visible outputs without influencing feature extraction or inference pipelines, thereby maintaining analytical consistency while introducing controlled behavioral modulation.

10.6.6 Monotonic Escalation Property

The DECEIVE engine enforces a monotonic escalation constraint across defensive states for all three modulation dimensions: latency, failure probability, and substitution rate. No

regression in deception intensity occurs during upward state transitions, and hysteresis-based demotion prevents oscillatory behavior under borderline classification outputs.

This monotonicity ensures predictable and stable adaptive behavior, enabling proportional escalation of adversarial friction without compromising containment guarantees.

11. Discussion

The results demonstrate that supervised behavioral modeling combined with state-aware deception enables stable threat classification alongside measurable adversarial friction. While offline evaluation achieved 90.18% accuracy and 0.8818 macro F1, structured evaluation accuracy decreased to 71.30%. This reduction reflects controlled distribution shift rather than instability, as the structured dataset intentionally introduced boundary overlap between HIGH and CRITICAL classes. LOW and MEDIUM discrimination remained stable, indicating robustness of the behavioral feature design.

Feature importance analysis confirms that semantic behavioral indicators particularly privilege escalation and exfiltration-related features dominate classification performance. Timing-based metrics contribute comparatively less, suggesting that operational severity in SSH environments is primarily defined by action semantics rather than execution speed.

Table 8 verifies that activation of adaptive deception policies does not alter classification accuracy, macro F1, or hard-trigger detection reliability. This invariance confirms architectural decoupling between observation and response layers. Deception modifies attacker-visible outputs without influencing telemetry capture or inference pipelines.

Quantitative escalation results demonstrate strictly monotonic increases in latency, failure probability, and output substitution across defensive states. The $93.5\times$ delay amplification and 38.4% failure rate under containment conditions represent measurable adversarial cost while preserving deterministic safety guarantees.

Limitations include reliance on structured evaluation scenarios and absence of long-term internet-facing deployment analysis. Future work will focus on extended real-world validation and sequence-aware behavioral modeling.

Future Work

Although the proposed Adaptive Self-Deceiving Honeypot demonstrates effective integration of supervised threat classification with a state-aware deception engine, several avenues exist for further refinement and extension.

A primary direction involves aligning model training more closely with runtime conditions. The current classifier is trained on full-session features, whereas deployment decisions are made on partial session windows. Future research should explore window-aligned training, progressive feature accumulation, and incremental prediction models that continuously update threat estimates as sessions evolve. Such approaches may significantly improve early-stage classification reliability and reduce initial misclassification bias.

Another important direction concerns improving discrimination at the boundary between HIGH and CRITICAL threat categories. Behavioral-only escalation sessions often overlap in feature space with lower-intensity CRITICAL sessions. Incorporating richer behavioral representations such as weighted event transitions, stage-aware indicators, or probabilistic severity modelling could enhance separation without sacrificing interpretability.

Refinement of the CRITICAL sub-level (C1–C6) mapping also presents a valuable opportunity. The current linear count-based scoring mechanism provides transparency but lacks semantic awareness of command intent. Future work may integrate rule-aware logic, command context modeling, or hybrid statistical–semantic scoring frameworks to improve fidelity in severity differentiation while preserving deterministic behavior for safety-critical escalation.

Expanding the feature space to incorporate temporal and sequential information represents another promising direction. Sequence-aware models, including recurrent neural networks, transformer-based architectures, or graph-based interaction modeling, could capture multi-stage attack progression and command ordering effects that are not represented in aggregate count features. Such models may enable more accurate modeling of attacker intent and structured behavior.

From an evaluation perspective, extending experiments beyond scripted attack profiles would strengthen ecological validity. Deployment in controlled internet-facing environments, red-team simulations, or the use of anonymized real-world attack traces would provide broader behavioral diversity and more rigorous validation of generalization performance.

Finally, hard-trigger mechanisms may be enhanced through semantic parsing, anomaly detection, or adaptive rule generation to improve robustness against obfuscated commands and novel attack variants. This would ensure continued reliability of safety escalation while maintaining flexibility against evolving adversarial tactics.

Collectively, these directions aim to evolve the system into a more sequence-aware, semantically grounded, and field-validated adaptive cyber deception framework capable of modeling complex adversarial behavior with greater precision and realism.

Conclusion

This study presented an adaptive self-deceiving honeypot architecture that integrates supervised behavioral classification with state-aware response modulation and deterministic containment enforcement. Using an 18-dimensional session-level feature representation and XGBoost-based multi-class modeling, the system achieved 90.18% accuracy and 0.8818 macro F1 under stratified evaluation, with consistent performance across LOW and MEDIUM threat categories.

Controlled comparative validation demonstrated that activation of adaptive deception policies does not degrade classification performance or safety guarantees. Hard-trigger detection and state-transition correctness remained at 100% across configurations, confirming architectural decoupling between inference and response manipulation.

Quantitative escalation analysis further showed strictly monotonic increases in injected latency, failure probability, and output substitution across defensive states. Under containment conditions, delay amplification reached $93.5\times$ relative to baseline, introducing measurable adversarial friction while preserving telemetry integrity.

Collectively, the results establish that adaptive deception can be implemented as a measurable and controllable defensive mechanism rather than a purely heuristic or simulation-driven strategy. The proposed DECEIVE framework demonstrates that supervised behavioral modeling combined with deterministic escalation provides both analytical transparency and operational containment reliability in SSH honeypot environments.

References

- [1] L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, Sep. 2002.
- [2] K. I. Iyer, “Adaptive honeypots: Dynamic deception tactics in modern cyber defense,” *International Journal of Science and Research Archive (IJSRA)*, vol. 4, no. 1, 2021, doi: 10.30574/ijrsra.2021.4.1.0210.
- [3] S. Dowling, M. Schukat, and E. Barrett, “Improving adaptive honeypot functionality with efficient reinforcement learning parameters for automated malware,” *Journal of Cyber Security Technology*, vol. 2, no. 2, pp. 75–91, 2018, doi: 10.1080/23742917.2018.1495375.
- [4] A. Pauna, A.-C. Iacob, and I. Bica, “QRASSH—A self-adaptive SSH honeypot driven by Q-learning,” in *Proc. 2018 International Conference on Communications (COMM)*, 2018, pp. 441–446, DOI:10.1109/ICComm.2018.8430173.
- [5] S. Touch and J.-N. Colin, “Asguard: Adaptive self-guarded honeypot,” in *Proc. 17th International Conference on Web Information Systems and Technologies (WEBIST)*, 2021, pp. 565–574.
- [6] S. Touch and J.-N. Colin, “A comparison of an adaptive self-guarded honeypot with conventional honeypots,” *Applied Sciences*, vol. 12, no. 10, p. 5224, 2022, doi: 10.3390/app12105224.
- [7] S. Veluchamy and R. S. Kathavarayan, “Deep reinforcement learning for building honeypots against runtime DoS attack,” *International Journal of Intelligent Systems*, vol. 37, no. 7, pp. 3981–4007, 2022, doi: 10.1002/int.22708.
- [8] E. Limouchi and I. Mahgoub, “Reinforcement learning-assisted threshold optimization for dynamic honeypot adaptation to enhance IoBT networks security,” in *Proc. 2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2021, pp. 1–7.
- [9] N. Jadav, N. Dutta, H. K. D. Sarma, E. Pricop, and S. Tanwar, “A machine learning approach to classify network traffic,” in *Proc. 2021 13th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*, 2021, pp. 1–6.

- [10] L. J. Möller, “An adaptive multi-layered honeynet architecture for threat behavior analysis via deep learning,” *arXiv preprint*, arXiv:2512.07827, 2025.
- [11] D. Reti, K. Elzer, D. Fraunholz, D. Schneider, and H.-D. Schotten, “Evaluating deception and moving target defense with network attack simulation,” *arXiv preprint*, arXiv:2301.10629, 2023.
- [12] M. A. Sayed, A. H. Anwar, C. Kiekintveld, and C. Kamhoua, “Honeypot allocation for cyber deception in dynamic tactical networks: A game theoretic approach,” *arXiv preprint*, arXiv:2308.11817, 2023.
- [13] A. Javadpour *et al.*, “A comprehensive survey on cyber deception techniques to enhance honeypot effectiveness,” *Computers & Security*, 2024, Art. no. 103792, doi: 10.1016/j.cose.2024.103792.
- [14] M. Y. Fatemi, W. A. Suttle, and B. M. Sadler, “Deceptive path planning via reinforcement learning with graph neural networks,” *arXiv preprint*, arXiv:2402.06552, 2024.
- [15] J. Chelliah and Y. Swamy, “Deception and lies in business strategy,” *Journal of Business Strategy*, vol. 39, no. 6, pp. 36–42, 2018.
- [16] P. Masters and S. Sardina, “Deceptive path-planning,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 4368–4375.
- [17] A. Price, R. Fraga Pereira, P. Masters, and M. Vered, “Domain-independent deceptive planning,” in *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2023, pp. 95–103.
- [18] Y. Huang and Q. Zhu, “Deceptive reinforcement learning under adversarial manipulations on cost signals,” in *Decision and Game Theory for Security (GameSec 2019)*, Lecture Notes in Computer Science, vol. 11836, Springer, 2019, pp. 217–237.
- [19] A. Lewis and T. Miller, “Deceptive reinforcement learning in model-free domains,” in *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 33, 2023, pp. 587–595.
- [20] Z. Liu, Y. Yang, T. Miller, and P. Masters, “Deceptive reinforcement learning for privacy-preserving planning,” in *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2021, pp. 818–826.
- [21] Y. Savas, M. Ornik, and U. Topcu, “Deceptive planning for resource allocation,” *arXiv preprint*, arXiv:2206.01306, 2022.
- [22] Y. Savas, C. K. Verginis, and U. Topcu, “Deceptive decision-making under uncertainty,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, 2022, pp. 5332–5340.
- [23] X. Qin, F. Jiang, M. Cen, and R. R. M. Doss, “Hybrid Cyber Defense Strategies Using Honey-X: A Survey,” *Computer Networks*, vol. 230, 2023, doi: 10.1016/j.comnet.2023.109776.

- [24] X. Qin *et al.*, “CGAN-Based Cyber Deception Framework Against Reconnaissance Attacks in ICS,” *Computer Networks*, 2024, doi: 10.1016/j.comnet.2024.110655.
- [25] P. Malhotra, “LLMHoney: A Real-Time SSH Honeypot with Large Language Model-Driven Dynamic Response Generation,” arXiv:2509.01463, 2025.
- [26] H. T. Otal and M. A. Canbaz, “LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems,” arXiv:2409.08234, 2024.
- [27] “SBASH: RAG vs Prompt-Tuned LLM Honeypots,” <https://doi.org/10.48550/arXiv.2510.21459>.
- [28] R. A. Bridges *et al.*, “SoK: Honeypots & LLMs — More Than the Sum of Their Parts?,” arXiv:2510.25939, 2025.
- [29] E. Alatawi *et al.*, “Honeypot-Driven Intrusion Detection Systems,” *Symmetry*, 2025.
- [30] D. S. Morozov *et al.*, “The Sweet Taste of IoT Deception: An Adaptive Honeypot Framework for Design and Evaluation,” *Journal of Edge Computing*, 2024.
- [31] D. Park and G. G. Dagher, “Adaptive Honeypot Allocation in Multi-Attacker Networks via Bayesian Stackelberg Games,” arXiv:2505.16043, 2025.
- [32] C. Y. Wu, Y. Wang, and Y. Liao, “Active Honeypot Guardrail System: Probing and Confirming Multi-Turn LLM Jailbreaks,” arXiv:2510.15017, 2025.
- [33] S. G. Singh, “Discovering SSH Attack Patterns using Cowrie Honeypot and K-Means Clustering,” *International Journal of Computer Applications*, vol. 187, no. 74, pp. 32–39, Jan. 2026, doi: 10.5120/ijca2026926253.
- [34] D. Sanjana, “Designing Autonomous Systems That Use Deception Techniques to Detect and Respond to Cyber Threats,” in *AI-Driven Security Systems and Intelligent Threat Response Using Autonomous Cyber Defense*, 2025, pp. 305–324, doi: 10.4018/979-8-3373-0954-5.ch011.
- [35] V. V. Ouwehand, “Adversarial Honeypot Detection and Deployment with Reinforcement Learning,” 2026.
- [36] Muntaha *et al.*, “AI-Powered Cybersecurity for Modern Threat Landscapes,” *Spectrum of Engineering Sciences*, vol. 3, no. 5, 2025.
- [37] H. Do Hoang, *et al.*, “Hawkeyes: An intelligent honeypot allocation strategy for cyber deception,” *Computer Networks*, 2025. doi.org/10.1016/j.comnet.2025.111982.
- [38] B. M. Dumisani, “Game theory: Cyber deception based on the redundancy of diversified honeypots,” *Computers & Security*, 2025. doi.org/10.1016/j.procs.2025.07.162.
- [39] M. Zambianco, *et al.*, “A proactive decoy selection scheme for cyber deception,” *Computers & Security*, 2025. doi.org/10.1016/j.cose.2024.104144.

- [40] Rui Wang, Changjiang Yang, Xiangdong Deng, Yinghai Zhou, Yuan Liu, Zhihong Tian, Turn the tables: Proactive deception defense decision-making based on Bayesian attack graphs and Stackelberg games, *Neurocomputing*, Volume 638, 2025, 130139, ISSN 0925-2312, doi.org/10.1016/j.neucom.2025.130139.
- [41] Kouremetis, M., Lawrence, D., Alford, R. *et al.* Mirage: cyber deception against autonomous cyber attacks in emulation and simulation. *Ann. Telecommun.* **79**, 803–817 (2024). <https://doi.org/10.1007/s12243-024-01018-4>
- [42] Rogers, A., Shade, T., Major, M., Johnson, C.K., Ferguson-Walter, K. (2026). The Tularosa Experiment: A Foundational Study for Cyber Deception. In: Zhu, Q., Lu, Z., Yu, P.L., Wang, C. (eds) Foundations of Cyber Deception. Advances in Information Security, vol 90. Springer, Cham. https://doi.org/10.1007/978-3-031-93867-2_6
- [43] Letychevskiy, O. Cognitive Networks, Their Properties and Applications in Attack Detection and Prevention Systems. *Cybern Syst Anal* **59**, 859–866 (2023). <https://doi.org/10.1007/s10559-023-00621-y>.